

Systems Concepts Digital Synthesizer operations manual and tutorial

by Gareth Loy

Center for Computer Research in Music and Acoustics
Department of Music
Stanford University
Stanford, California 94305

Abstract

This document attempts to shed light on the operation of the Systems Concepts Digital Synthesizer, designed and built by Peter Samson. While it treats of this machine specifically, the subject is presented from a broad signal processing perspective, and as such provides a good general introduction to special purpose hardware for real-time digital synthesis. The manual begins with a general discussion of the architecture of the machine as a whole, then goes into more detail about command and sample data structures, the workings of the processing elements, and finally, a discussion of the command assembler and other software created at Stanford. At the end are appendices on various topics ranging from tutorials on aspects of signal processing to summaries of important tables from the text, and examples of simple programs using the Stanford software. Great lengths have been taken to make the text as clear and comprehensible as possible to the widest range of readers.

Acknowledgments

The existence of this document is entirely due to the patience, teaching and editing I have received over the course of its writing from Andy Moorer and Mark Kahrs. Additional thanks go to every person who has helped to build the software support, including Ken Shoemake, Bill Schottstaedt, Mike McNabb, Peter Nye, Tovar, John Gordon, Julius Smith, John Chowning, John Grey and Loren Rush. Lastly, thanks are due to those who have used the Synthesizer, whose comments and suggestions form the only known cure for bugs.

Credits

The section, "A Little Digital Filtering theory (...never hurt anybody)" was written by Andy Moorer. Marc LeBrun wrote the discussion of the linear congruential uniform noise applications. Ken Shoemake wrote the section on "Cold starting the Box". Julius Smith wrote the discussion of how to apply exponential envelopes in generators.

T A B L E O F C O N T E N T S

	SECTION	PAGE
Section 1	Introduction	
	1.1 The nature of the problem	1
	1.2 Reading this manual	1
Section 2	General definitions	
Section 3	Ticks	
Section 4	Tick time requirements of processing elements	
	4.1 Generator tick time	6
	4.2 Modifier tick time.	6
	4.3 Combined tick times	6
	4.4 General formula	7
Section 5	Introduction to the processing elements	
	5.1 Generators	8
	5.2 Modifiers	8
	5.3 Sum memory	9
	5.4 Delay memory.	9
	5.5 Delay units	9
	5.6 Command mnemonics	9
Section 6	The flow of data	
	6.1 The Connection....	12
	6.2 I/O bus stream	12
	6.3 Command stream.	12
	6.4 Read stream and Write stream	13
	6.5 DAC stream	13
	6.6 It's a Stream machine...	14
Section 7	Sum memory	
	7.1 This pass - last pass division.	15
	7.2 Generator - modifier division	16
	7.3 Addressing sum memory	16
Section 8	Introduction to Generators	
	8.1 Generator parameters.	18
Section 9	Generator running modes	

	9.1	INACTIVE	20
	9.2	G_PAUSE	20
	9.3	G_WAIT	21
	9.4	x_RUNNING.	21
	9.5	DAC_WRITE	22
	9.6	READ_DATA	22
	9.7	sample interleaving for read/write data	22
	9.8	WRITE_DATA	23
Section 10		Oscillator processing	
Section 11		Oscillator modes	
	11.1	SINE	27
	11.2	SAWTOOTH	27
	11.3	SQUARE.	28
	11.4	PULSE_TRAIN	28
	11.5	SIN_FM	28
	11.6	SUM_OF_COSINES.	29
	11.7	A note on band limiting	29
Section 12		Envelope processing	
	12.1	EXPONENT and RATE	30
	12.2	ASYMPTOTE	30
Section 13		Envelope modes	
Section 14		More on generator processing - alternate modes	
	14.1	SWEEP	33
	14.2	FREQUENCY	33
	14.3	ANGLE	34
	14.4	NCOSINES and SCALE	34
	14.5	FM	35
	14.6	SUM_MEMORY.	35
	14.7	RATE EXPONENT and ASYMPTOTE	35
Section 15		Generator magic numbers	
	15.1	Specifying frequencies to generators	36
	15.2	More about the FREQUENCY parameter	37
	15.3	SWEEP scaling	37
	15.4	FM scaling	38
Section 16		Modifiers	
	16.1	Modifier parameters	41
	16.2	Read terms	41
	16.3	Coefficient terms	42
	16.4	Scale terms	42
	16.5	Running terms	42
	16.6	Mode and Function	43

	16.7	Sum memory	43
	16.8	Modifier arithmetic	43
	16.9	Modifier this pass timing considerations.	44
Section 17	Modifier modes		
	17.1	Inactive	46
	17.2	Mixing	46
	17.3	Integer mixing	46
	17.4	Amplitude modulation	47
	17.5	Four quadrant multiply	47
	17.6	Latch	47
	17.7	Signum	48
	17.8	Minimum	48
	17.9	Maximum.	48
	17.10	Threshold.	48
	17.11	Zero-crossing pulser	49
	17.12	Uniform noise.	49
	17.13	Triggered uniform noise	50
	17.14	Invoke delay unit.	51
Section 18	Modifier filtering algorithms		
	18.1	One pole	52
	18.2	One zero	52
	18.3	Two poles.	53
	18.4	Two zeros.	53
	18.5	Two poles COEFF0 variable.	53
	18.6	Two poles COEFF1 variable.	53
	18.7	Two zeros COEFF0 variable.	54
	18.8	Two zeros COEFF1 variable.	54
	18.9	A little digital filtering theory	54
Section 19	Delay Units - Delay Memory		
	19.1	Delay line mode	62
	19.2	Table lookup mode	62
	19.3	Table lookup - rounded	62
	19.4	Other phenomena	63
Section 20	Instruction and command structure		
Section 21	Processing element commands		
	21.1	Miscellaneous command	66
	21.2	Delay command	67
	21.3	Timer command	67
	21.4	Ticks command	68
Section 22	Synthesizer instructions		
Section 23	The Stanford software		

	23.1	Overview	71
	23.2	Low level routines	71
Section 24		Low level allocation routines	
	24.1	GET	73
	24.2	GIVE	74
	24.3	Processing element tables.	75
	24.4	DECODE.	76
	24.5	RELATIVE	76
Section 25		Command assembly	
	25.1	Bind	77
	25.2	BIND_FIELD.	78
	25.3	SET_MODE	79
	25.4	SET_FIELD	79
	25.5	Assembler tick and pass counters.	80
Section 26		Stream output	
	26.1	INITIALIZE	82
	26.2	FLUSH	82
	26.3	SET_OUTPUT	82
	26.4	SET_PROCEDURE	83
Section 27		Disassembly - Reassembly	
	27.1	DAB	84
	27.2	UNDAB	85
	27.3	MERGE	85
Section 28		How to crank up the Samson box	
	28.1	General considerations	86
	28.2	Cold starting the box.	86
	28.3	Cold starting from a real-time job	89
	28.4	Sample code for a SAIL program on the KL-10	91
Appendix 1		An introduction to pipelining	93
Appendix 2		Time division multiplexing	95
Appendix 3		Basic number theory	97
Appendix 4		More about processing speed vs. capacity	101
	4-1	Speed of processing	101
	4-2	Ticks per pass.	101
	4-3	Processing vs. update ticks	102
	4-4	Numbering of elements and ticks.	102
Appendix 5		SAIL examples	103
	5-1	Generator example	103

	5-2	Modifier example.	108
	5-3	Delay line example	109
	5-4	Writing your own procedures	110
Appendix 6		Generator field and mode tables.	112
	6-1	Generator parameters.	112
	6-2	Generator run modes.	112
	6-3	Oscillator modes	113
	6-4	Envelope modes	113
Appendix 7		Modifier field and mode tables	114
	7-1	Modifier parameters	114
	7-2	Modifier procedures	115
Appendix 8		Delay unit field and mode tables.	116
	8-1	Delay units: timing	116
Appendix 9		Reserved Words	118
Appendix 10		Error messages	122
Appendix 11		References.	122

Section 1

Introduction

1.1. The nature of the problem

The Systems Concepts Digital Synthesizer is a special purpose digital computer, designed for high quality audio signal generation and processing. It produces streams of binary numbers which represent the waveform of the desired sound. The numbers that make up these streams are individually called samples and represent the instantaneous value of the signal in much the same way that an individual frame of a movie is the instantaneous value of the represented motion. These samples can then be converted by the synthesizer into analog voltages that can be fed to a sound system.

There is a need for extreme speed in calculating the samples which can be appreciated if it is realized that according to the Nyquist Sampling Theorem, one cannot represent a frequency higher than $1/2$ of the rate at which the samples are produced (called the sampling rate). This means that for reasonably good hi-fi sound, where the frequency response might go up to 15 kHz or beyond, the machine must be capable of at least a 30 kHz sampling rate per channel of sound. This can be easy or difficult to do depending on how much computation goes into each sample. But typically, each sample is made up of great amounts of computation such that the speed limits of most modern computers can be quickly surpassed, especially considering that production of interesting music by computers typically requires a great deal of sophistication in the models used to generate the sounds. So we see that we must be clever in how we structure the computations in order to get a good fast sampling rate but still to have enough time to compute each sample. The Samson box is a study in this kind of cleverness.

1.2. Reading this manual

This manual starts out assuming an elementary knowledge of the basic computer science concepts of pipelining and time division multiplexing. If these terms are unfamiliar to you you may want to first read appendices 1 and 2 which are designed to impart an understanding of these things to the (not quite total) novice user. If you would like to review basic digital number theory, read appendix 3.

The later sections on calling the Samson box from a SAIL environment and the subsequent program examples are designed to be read by those already familiar with the SAIL language. SAIL is an extension of the ALGOL language which is used at the Stanford Artificial Intelligence Lab. Generally, if you are only interested in the basic processing of the box read until section 23.

Section 2

General definitions

Before proceeding let's define some terms. These definitions are elaborated in later sections of this manual.

Processing elements:

The Samson box has two basic active parts, or processing elements. One of them is called a generator, (see section 8) the second is called a modifier (see section 16). There are 256 generators and 128 modifiers.

The processing elements communicate with each other through a scratchpad read-alter memory called sum memory which is used to store the output from, and input to the processing elements. There are 256 locations in sum memory. See section 7.

Delay units are a special kind of processing element. They must work in conjunction with modifiers. They can reference up to 65k of 20 bit words of memory (of which Stanford has 48k words). It is called delay memory. This is for reverberation and general table lookups. See section 19.

Timing: The time that it takes a processing element to do its thing is called a tick. A tick is the smallest counting unit in the box, similar in concept to the tick of a clock. As in any synchronous chain of events, the whole can not run faster than its slowest component, so likewise the tick is the amount of time necessary for the slowest micro-process to complete its operation. See section 3.

The time, measured in ticks, that it takes to go once through the complete processing cycle of the box is called a pass. The time required for a pass is therefore the sum of ticks it takes.

At the end of a pass we have calculated one instantaneous value of our waveform. This value is called a sample. A pass is therefore not just the cycle time of the process we are running, it is also the time it takes to produce a sample.

The term sample period is used to refer to the amount of time in seconds it takes to produce a sample. So the sample period and the pass period will always be equal. The distinction between the two terms is merely that one is measured in ticks, the other in seconds.

One final term: sampling rate is the ratio of the number of samples produced per second. It is the reciprocal of the sample period.

Section 3

Ticks

The tick, as I said, is the basic cycle time of the Samson box. There are three classes of tick: processing, update and overhead. All kinds of ticks take the same amount of time, 195 nanoseconds, which is called a tick-time.

When the processing elements are running, processing ticks are being used.

When new values are being entered into the processing element parameters, update ticks are being used.

These are the two basic tasks of the synthesizer. There can be a variable number of ticks within a pass. If you need a lot of processing elements, you need a lot of processing ticks, if you need to change a lot of parameters, you need a lot of update ticks.

Overhead ticks are a kind of processing tick, which are necessary because of the pipelining of the processing elements. There are always eight overhead ticks per pass. Their function is discussed more in appendix 1.

Now let's take an example that will show how processing and update ticks interact.

Since a pass is the sum of processing and update ticks (and overhead ticks), we must anticipate the maximum number of processing element parameters we will want to change per pass. It takes one tick-time to update one parameter in a processing element. That number is then the total update ticks required.

Next we decide how many processing elements we want to run and figure out how many processing ticks this will require. Each running processing element takes 8 tick-times, but because of the pipelining of the processes (i.e., the overlapping of several calculations at once) the actual number of processing ticks is one for generators and two for modifiers. See the next section for more about this.

But in any case, we figure out how many it will take for the processing we desire, and then add all the three kinds of ticks together. This number is called the total ticks. We forward the total ticks thus calculated to the box on a command and the length of a pass is set accordingly as:

$$\text{pass period} = \text{total ticks} * \text{tick-time.} \quad \text{Equ. 1}$$

There is a reciprocal relationship between the pass period and the sampling rate: the longer the pass period, the slower the sampling rate. Thus, the more you have the synthesizer do, the longer it takes, and the slower it runs. The sampling rate determines (according to the Nyquist theorem) the highest frequency it can produce, specifically:

$$\text{highest frequency} = \text{sampling rate}/2.$$

Equ. 2

Therefore it can be seen that the size of the total ticks number must often be a compromise between the desire to maintain a sufficiently high sampling rate to guarantee a minimum high frequency response, versus a desire to have either a lot of processing happening, and/or a lot of change in the process.

Now besides setting up the total ticks, we also need to specify how many of these are update and how many are processing ticks. This is done by just passing the processing tick number to the box on a command, and the box figures out the update ticks as:

$$\text{update ticks} = \text{total ticks} - (\text{processing} + \text{overhead ticks}).$$

Equ. 3

So in effect, the number of update ticks per pass is what is left after processing and overhead ticks.

When the box runs, it first executes all the processing ticks, then the overhead ticks, finally the update ticks are run in the remaining time. Notice that this means that on the very first pass, the box may be operating on data left over from the last time somebody used it. Care must be taken that the process is initialized in such a way as to avoid the consequences of this.

Now let's try to develop a model for what this looks like in practice. Let's take as an example a generator that has two user-settable locations. One location contains the number that represents the frequency. The other location contains a number which changes the number in the frequency location by adding its value to the number in the frequency location on every pass, thereby raising or lowering the frequency uniformly by some amount or "slewing" it, producing a glissando. We will diagram it this way:

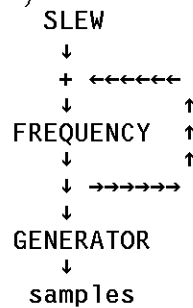


Fig. 1

On every pass the number in FREQUENCY is sent to the GENERATOR which produces the sample passed as the output. But the FREQUENCY number is also piped around and added into the number coming down from SLEW. The sum of FREQUENCY and SLEW replaces the old value in FREQUENCY. But SLEW doesn't change, it's a constant. You can see that this thing can run forever by itself. On each pass a new frequency is formed and a new sample is shipped out.

Since we'd like to be able to change its values from time to time, let's declare that there will be two ticks per pass, one processing tick, and one update tick (leaving out overhead ticks for a moment). As we begin looking at the processing stream, we notice it's already been running and that the last output was 1000 and SLEW had 1 in it. If this went on as is, we would send succeeding values of 1001,1002,1003... etc. to the GENERATOR. But let's say that we're tired of this and want to change it, so we'll put 3 in SLEW at our first chance (where the first vertical arrow is), and we'll change FREQUENCY to 500 at our next chance (at the second arrow). The two rows SLEW and FREQ. in the next figure show the progression of values of the two settable locations in our generator. The bottom line shows what frequency the generator is actually producing on that pass.

	<--- pass -->		<-- pass --->		<--- pass -->		<--- pass -->	
	proc	upd.	proc	UPD.	proc	UPD.	proc	upd.
Slew	1	1	1	3	3	3	3	3
Freq.	1001	1001	1002	1002	1005	500	503	503
Out	1000		1001		1002		1005	503 ...

Fig. 2

It's not easy, but fortunately we don't have to think about this level of detail very often.

One last word about processing and update ticks. Processing elements make references to sum memory and delay memory during processing ticks, not update ticks. Update ticks only refer to data coming in from outside the Samson box.

Section 4

Tick time requirements of processing elements

4.1. Generator tick time

Each running generator requires one processing tick; the length of the generator pipeline is 8 ticks.

So if we had, say, 11 generators running, and assumed for the moment that there were no update ticks, how many ticks would it take before we got a complete set of valid results for all the generators? That is to say, when would the first pass end? It would end after 19 ticks, right? If not right, go read the part about pipelining in appendix 1. When would the next pass and all subsequent passes end? Right,... after 11 more ticks. So the formula for processing ticks required by each running generator is:

$$\langle \# \text{ of running generators} \rangle + 8. \quad \text{Equ. 4}$$

4.2. Modifier tick time

Modifiers do general numerical and logic processing. As the name implies, their primary function is to modify values that have been created by other processes. They are supplied with two inputs and one output to sum memory. See section 16. Since modifiers have to do two memory references to get input data from sum memory where generators do but one (see section 8) they require two processing ticks.

The length of the modifier pipeline is also 8 ticks. So say we had 7 running modifiers with no update ticks: how long until the first complete set of valid numbers comes out? The answer is 22. So, the tick requirement for modifiers is:

$$2 * \langle \# \text{ of running modifiers} \rangle + 8. \quad \text{Equ. 5}$$

4.3. Combined tick times

Now let's take an example where we have both modifiers and generators running simultaneously (still without update ticks). We must first know that there are really two processors, one of which represents the modifiers and the other the generators. These are separate nests of electronics that run essentially independently of each other (although they are actually in lock step at the level of the tick).

How many processing ticks for the first pass would be required if we have, say, 11 running generators and 5 running modifiers? The correct answer is 19 ticks. The tick requirement for the first complete set of modifier output is 18, but that for generators is 19. Since they are parallel processes, the modifiers can do their whole thing in 18 ticks while the generators are also running.

So we see that the box must not consider that it has completed a pass until the number of ticks representing the longest processing chain is done. Another way to say this is that the pass period is the maximum of the generator tick time vs. modifier tick time.

4.4. General formula

So the official formula reads as follows, and I quote: "The number of processing ticks per pass is the maximum of: the number of generators used; twice the number of modifiers used." ¹ Or,

$$\text{total processing ticks} = 2 * \langle \# \text{ modifiers used} \rangle \text{ MAX } \langle \# \text{ generators used} \rangle + 9. \quad \text{Equ. 6}$$

The Delay units have their own formula for tick requirements which is not shown here. See Appendix 8.

For a further discussion of timing criteria, see Appendix 4.

¹ From the [Specifcation].

Section 5

Introduction to the processing elements

As you have seen, the Samson box has two basic types of processing elements: generators and modifiers. In fact, there are several other components which are described below. In the general use of the term, all of these components will be referred to as processing elements.

5.1. Generators

The generators perform two functions: periodic waveform generation and dynamic amplitude control of the generated waveform. The portion that performs the waveform generation is called the oscillator part, the other which handles the amplitude control is the envelope part. Generators can produce sine, square and saw-tooth waveforms, as well as pulse train and equal-amplitude sum-of-cosines (band limited pulse train) waveforms. There is an input port to the frequency term of the oscillator part from sum memory that allows for frequency modulation by the output of any other processing element. In addition the frequency can also be swept linearly.

The envelope side has facility to shape amplitude using either linear ramps or exponential curves.

There are 256 generators, and they can all be running simultaneously.

Just what the generator does is dependent on the its mode. The mode and the values acted upon are set during update tick time.

5.2. Modifiers

Modifiers can act as filters, amplitude modulators, four-quadrant multipliers (signed multiply), mixers and triggerable uniform noise generators (white noise and random number generation). They can also do signal latching (sample and hold), sign testing, signal zero-crossing detection, and minimum or maximum testing.

Modifiers are also used to pass data to and from the delay units for reverberation or table lookup.

The box contains 128 modifiers all of which can be running at once. Again, the mode and all values are set during the execution of update ticks.

5.3. Sum memory

Information is passed among generators and modifiers through a "scratch pad" sum memory. The effect of individual sum memory locations is to form the sum of the samples passed as output from the various processing elements that address them. On the pass after the sum is formed, other processes can then reference this result as input for further processing. Or, the sample can be output to the DACs, or written back to the controlling computer, etc. Thus it can be thought of as a kind of telephone switch-board because it is through the sum memory that the data put out by one processing element is accessed by other processing elements. Samples are written to sum memory and retrieved from it during the execution of processing ticks.

5.4. Delay memory

There is a memory of up to 65K words available as an optional feature (of which the Stanford prototype has 48K words). Delay memory has two functions: It can be used to simulate delay lines for reverberation and filtering, or it can be used to store precomputed tables, such as time-domain waveforms or mathematical functions not otherwise available in the synthesizer (such as square root tables, etc.).

5.5. Delay units

Delay units are used to read from and write to delay memory. They pick up values to be written in delay memory from modifiers that have been put in a special mode that connects them to a specific delay unit. The unit also reads values from delay memory and sends the retrieved value back to the controlling modifier.

There are 32 delay units, and they can all be running at once.

All references to delay memory must be through delay units. Each unit stores information about the block of memory it accesses and the index into the memory.

It is possible for any unit to access any part of delay memory, hence it is possible to have more than one unit overlap the same section.

5.6. Command mnemonics

As we have seen, processing elements have internal fields, or registers which store their current state, i.e., whether are turned on and off and what values they are operating on. These things are all set up or changed by issuing commands to the Synthesizer. There are three parts to such commands: they specify 1) which processing element is addressed by the command, 2) which field

or parameter of the processing element to change, and 3) the new value to give it. The method of passing the commands to the Synthesizer as well as the details of the structure of these commands is given later (see section 6 and 20).

Throughout the rest of the manual, where a particular field of a processing element is described for the first time, two terms will be given, the first is the (usually not very) mnemonic term used in the [Specification]. The second (slightly more) mnemonic term is both a description of the actual function of the parameter, as well as a symbolic name for it which can be passed to the command assembler described later (see section 23) to specify the particular field. This second set of terms will be used throughout the rest of this document to refer to these parameters. They will always appear capitalized. The first set is given only for cross referencing to the [Specification].

DATE!

DRAFT

BLKDIA.XIP[DOC,MUS] goes here.

Section 6

The flow of data

6.1. The Connection...

The synthesizer is designed to require the services of a controlling computer to accomplish several tasks which fall into five categories. From here on we'll call the controlling computer the central processing unit, or CPU. For the complete interface to the synthesizer, the CPU must be able to: (1) control the overall operating status of the synthesizer (for instance, to start and stop the whole machine); (2) to configure the running modes of the various processing elements and feed them values to operate on; (3) send sample data to the synthesizer for processing; (3) receive processed sample data back from the synthesizer; (4) send and receive diagnostic information for debugging; (5) there is also the ability to load delay memory directly from the CPU as well as via modifiers and delay units. To accomplish this communication, there are several connections, called "paths" or "streams", between the CPU and the synthesizer, each of which have specific functions described next.

6.2. I/O bus stream

First, we must have a way to set the general status of the box. This has two main functions among a variety of others: (1) to start and stop the entire synthesizer and (2) tell it were in the memory of the CPU it is to look for commands to configure the individual processing elements, and also possibly for read or write data. This is called the I/O (for in/out) stream because it is bidirectional. This means that the synthesizer can also send information back to the CPU over this line when it encounters errors in processing, data, or if the computer has requested a diagnostic readback of internal synthesizer computations for debugging purposes. It is a relatively slow stream in comparison to the others. This stream runs basically asynchronously to the synthesizer and the CPU insofar as the one can interrupt the other at any moment.

6.3. Command stream

Next, we have a stream from the CPU to the box to send instructions to the processing elements inside the synthesizer. The content of this stream will be commands to the elements to set their modes, where they connect to sum memory, and the actual data to be put in the parameter fields for such things as the frequency of generators, etc. This synthesizer reads the commands received on this stream during update tick time.

The command stream is what is known as a direct memory access channel or DMA. It is a direct, high speed link from the synthesizer to the memory of the CPU. The way it works is that the CPU first finds a free block of its core memory, then fills it with update commands for the processing elements in the box. Then it uses the I/O stream to pass the address of this block of commands to the box. Then the box, during the processing of update ticks, reads in this block of commands one at a time until exhausted. The synthesizer then interrupts the CPU with the fact that it is done and requests more input.

6.4. Read stream and Write stream

A very crucial feature of the synthesizer is its ability to read in and write out sample data to and from the CPU. So, for instance, we could reverberate a sound read in from an analog to digital converter, which could come directly from a microphone, etc. As for writing samples, it would allow us to take the output of the synthesizer and analyze it with any available digital technique, or to do further processing on it with some other process not available on the synthesizer. A very important aspect of this capability is that it is a way of expanding the computational ability of the synthesizer. For instance, if the synthesizer just doesn't have enough processing power to realize a sound, or piece in one run-through, it can be done in as many passes as desired by overlaying new samples with the old ones that have been written out from the previous pass, for a theoretically infinite complexity.

The read and write streams are also DMA channels, and the structure is identical to the command stream described above. We have a problem, however: notice that sum memory can only be addressed by processing elements, the DMA cannot reference sum memory itself. The solution is to have a special mode for one of the processing elements such that it links up with the DMA to pass the information to sum memory where it can then be accessed by the other processing elements. The generators were chosen for this task (the implementation is discussed later).

To return sound samples back to the computer we have the same problem. Only processing elements can read sum memory. So again the generators have another special mode that reads sum memory and passes the sample data to the DMA channel which writes it into the CPU's memory.

6.5. DAC stream

Lastly, how do we get samples from out of the box to the DACs? Again there is a special mode for the generators which causes them to read sum memory and pass the data to a specific DAC. The synthesizer can address 16 separate DAC outputs, of which the Stanford prototype has 4.

6.6. It's a Stream machine...

This is an appropriate time to discuss what a "stream" is. Like a real stream, each active path to the box expects a constant flow of information about what it is supposed to be doing. The command path is always active. If there is a point in the process where no new commands need to be given to the synthesizer, it must be told not to expect any commands for a specified period of time. This period of time can either be specified relative to the first tick or from the current tick. Thus you can set up some state in the box and tell it to continue doing what it is set up to do. The way to do this is to pass a LINGER command (See page 79). This command is passed to the box in the command stream and it instructs the box not to process any more update commands until it has finished so many passes.

When the synthesizer runs out of processing element commands by exhausting the buffer of them in the CPU's memory, it generates an interrupt to the CPU over the I/O stream, telling it to inform the synthesizer where to find the next buffer of commands so it can continue.

If the CPU doesn't give it a new buffer before the Synthesizer runs out of commands to process, then the situation is known as command underrun. Under these conditions, the box continues running, doing whatever it was doing. This can be ok, or not depending on various things. If, for instance, the amplitude term of a generator was set to climb, and no command comes along to stop it, it'll continue to get louder and louder and louder... You get the picture. If, however, the configuration of the synthesizer is stable, then this is a perfectly reasonable way to leave it. Technically, one should always be feeding the command stream LINGER commands to keep it happy, but the Synthesizer is perfectly ok running a static patch.

Now in the case of read and write data, the situation is somewhat different. If, for instance, the synthesizer is reading samples from the CPU's memory, and it runs out of its current buffer of samples, to go on processing would be meaningless, since the process depends upon getting this input. So, if after interrupting the CPU to try to get another buffer for read or write sample data, no such buffer is forthcoming, the synthesizer will stop processing until the buffer is provided. This would be termed read/write data underrun.

There is also the possibility of command overrun. This presumes that some commands were missed somehow. One of the effects that this could have is to cause the next LINGER command to request waiting for a time that is so far in the future that the box thinks it is a request to run in the past (because the pass counter has wrapped around). The Synthesizer has a trap for this condition, but other instances of command overrun are not detectable.

Section 7

Sum memory

Sum memory is the place where samples are stored that are the output of processing elements and are to be the input to other processing elements. It is necessary that a value be read into a location in sum memory before it can be read by any other processing element. There are fields in the processing elements that specify which sum memory locations they are to use.

It's called sum memory because any value added in during a pass is added to the contents already there from the beginning of that pass. There are some special circumstances whereby modifiers can have their output overwrite the sum memory location, instead of add to it.

7.1. This pass – last pass division

Sum memory is divided in two basic ways, producing four quadrants. One of the divisions is between "this pass" sum memory and "last pass" sum memory. The active processing elements on the current pass all write into this pass sum memory and read from last pass.

This means that all the outputs of any given pass are in place before any reading of sum memory is done. After the processing of all the outputs has taken place (which, you should remember, is the definition of a pass), this pass sum memory is frozen and becomes last pass sum memory. This is nice in that it means that one doesn't have to worry about the order of computation of the processing elements inside of a pass: by the time you get around to reading the output of any processing element all the outputs of all the processing elements are available, ensconced in last pass sum memory.

However, there is a special feature that modifiers can also read from this pass sum memory (not to make things too confusing or anything), in which case the ordering of the modifiers inside a pass does become critical. If, for instance, the modifier that is trying to read a this pass sum memory location is executed before the one that is supposed to be supplying the number, you get zeros, or garbage, depending on what else is happening.

Please notice an implication of the separation in time by one pass of reading and writing into sum memory. Whereas the processing order within a pass is not critical (with the exception above noted), the processing between passes is critical if you are chaining a bunch of processing elements together. Say maybe you want to FM modulate an FM modulated generator, meaning that you have the output of one generator affecting the frequency of another generator affecting the frequency of yet another generator. This setup is yet another pipeline! So you must be sure to start the processing elements in order. Pass 1: the three generators start, and from generator 1 comes a valid number, but 2 and 3 produce nonsense; pass 2: generator 2 reads generator 1's output and gives a valid result; pass 3: finally the valid results have trickled through the system.

This means that there are a couple of passes where we might be producing garbage unless care is taken.

The last pass and this pass halves of sum memory alternate, one set being the current "this pass", then becoming "last pass". Before it again becomes this pass it is erased so that previous values don't hang around. Therefore reading a sum memory location without writing in it first will always return zero.

7.2. Generator – modifier division

The other basic division of sum memory has to do with which kind of processing element writes where.

One half of "this pass" sum memory is dedicated to output from generators. That means the generators have 64 locations in this pass sum memory that they can write into. The other half of this pass sum memory (64 locations again) is dedicated to the output of modifiers. For this pass sum memory this distinction remains firm. However, either type of processing element can read any location (128 of them) in last pass sum memory.

The modifiers have the ability, you recall, to read from this pass sum memory as well, however they are restricted to reading from the modifier quadrant of this pass sum memory. That makes a total of 192 locations that are available for modifiers to read from on a pass. Again, if a modifier is reading from this pass memory, the ordering of the units must be done carefully so that one is assured that the data is present in memory when you want to read it. This requires keeping track of the extensive pipelining in the machine, which we will not go into here. See page 44.

7.3. Addressing sum memory

Here's what the processing elements have to communicate with sum memory: each generator that is running is given a location in sum memory into which it deposits its generated sample. More precisely, there is a register in every running generator which contains a word that is the address of a location in sum memory into which that generator is to put its output. There are mnemonics which we will use for these various registers within the processing elements. In this case, the register in the generator that stores the sum memory address is called (appropriately enough) SUM_MEMORY. Generator is said to have an output port to sum memory. Generators likewise have one input port to read samples from sum memory called FM. This means that FM is a location in the generator which contains a word which is an address of a location in sum memory from which the generator is to read sample data.

The modifiers have one port to write, also called SUM_MEMORY (the assembler can make the distinction of whether we are referring to the SUM_MEMORY location in a modifier or generator as will be explained later), and two ports to read from sum memory called A_IN and B_IN.

The delay units have no direct port to the sum memory, but must be accessed by the modifiers, which pass data to the delay units and read data from them. The modifiers then transfer data to sum memory.

Section 8

Introduction to Generators

Generators produce periodic waveforms of various shapes and flavors and they control the amplitude of the waveforms through time. Each generator is divided into two parts: the oscillator is the side that makes the waveform; the the envelope side can dynamically scale the amplitude of the waveform.

The generators produce waveforms with envelopes by two basic methods 1) logical testing of a monotonically increasing function of time (as in the case of pulse train and square wave) or 2) table lookup (as in the case of sine waves and exponential envelopes). More on this later, but for a quick example, the most common use of generators is sinusoid generation. The oscillator generates an index into a sine function table which it then looks up. There are several things it must determine before it knows where to look in the table, most obviously what the frequency is, the phase, etc., etc.

The information the generator needs to produce its waveform is passed to it on update ticks. The values sent are stored in locations in the generators, from whence they are operated on by the generator during processing tick time to produce the waveform. Generators can receive one new parameters per update tick. Depending on what mode the generator

Some generator locations have different meanings and uses depending on what mode the generator is in, as will be shown in detail later.

8.1. Generator parameters

The generators have physical hardware registers that keep all the parameters of the signal being produced as well as places to store the running terms. In the tables below, two names for each parameter are given where the first is the (usually not very) mnemonic term used by Systems Concepts in the hardware Specification sheet (these are under the column marked Id). The second is a (slightly more) mnemonic giving the functional name of a parameter (denoted by the column marked Definition). This second term attempts to be both a description of the actual function of the parameter, as well as a symbolic name for it which can be passed to the command assembler described later (see section 23) to specify the particular field. This second set of terms will be used throughout the rest of this document to refer to these parameters. The first set is given only for cross referencing to the [Specification]. The third entry, the Size column, tells how many binary bits the field consists of. The Type column says whether the number is: Signed, Unsigned, Different in different modes or an Address. The notations in the Function column are elaborated more thoroughly later.

Id	Size	Type	Definition	Function
GO	20	S	SWEEP	oscillator frequency sweep rate
GJ	28	S	FREQUENCY	oscillator frequency
GK	20	D	ANGLE	oscillator angle
GN	11	U	NCOSINES	number of cosines to be summed
GM	4	U	SCALE	binary scale of sine or sum of cosines
GP	20	S	RATE	envelope rate of change
GQ	24	S	EXPONENT	envelope current value
GL	12	U	ASYMPTOTE	asymptote (DC offset of the envelope)
GSUM	6	A	SUM_MEMORY	sum memory location into which output is added
GFM	7	A	FM	sum memory address from which frequency modulation data is taken
GMODE	10		MODE	generator mode, including run mode and type of oscillator and envelope processing (see list of modes on page 20).
			OSC_MODE	sets oscillator field of SUM_MEMORY without altering run mode or envelope mode.
			ENVELOPE	sets envelope field of SUM_MEMORY without altering run mode or oscillator mode.

Fig. 3

Section 9

Generator running modes

The specification of the mode of a generator is broken down into three categories: 1) the mode of the oscillator part, 2) the envelope part and 3) the running status of the generator as a whole. This next table is a list of the third category of modes. The modes specific to the oscillator and envelope parts will be given later.

Definition	Osc. run?	Envelope run?	Add to sum?

G_INACTIVE	no	no	no
G_PAUSE	no	no	no
G_WAIT	yes	no	no
x_RUNNING:			
A_RUNNING	yes	yes	yes
B_RUNNING	yes	yes	yes
C_RUNNING	yes	yes	yes
DATA_READ	no	yes	yes
DATA_WRITE	no	no	no
DAC_WRITE	no	no	no

Fig. 4

9.1. INACTIVE

mode freezes both the oscillator part of the generator and the envelope side. Actually, the generator continues to be processed, but the changing of parameters is inhibited and it produces no output.

Please note that making a generator INACTIVE does not clear its fields. It retains the last values it had before going inactive. In fact, no field in any processing element except sum memory is ever cleared except by explicitly loading new values. For this reason it is important to set all the relevant parameters of a generator before you turn it on, else you might be subject to a certain form of astonishment.

9.2. G_PAUSE

freezes both the oscillator and the envelope as does INACTIVE, but makes it easier to restart. There is a special command called "clear all pause bits" (see page 79) that will make all pausing generators run again with only one command, whereas restarting generators in INACTIVE mode

requires an individual mode setting command for each generator. This is useful if, for instance, a bunch of generators were running, you want them to stop, but you don't want to have to send them all individual mode change commands to crank them up one by one which would gobble update ticks. This way also you are guaranteed that they will all restart later with the same phase and envelope positions as where they left off.

9.3. G_WAIT

continues running the oscillator side but turns off the envelope side and stops output. This is to let the oscillator keep in phase with any other generators that may be running concurrently. It can therefore reenter later and still be in phase synchronization with the rest of the world. There is also a "clear all wait bits" command (see page 79) later that encourages the use of G_WAIT mode.

9.4. x_RUNNING

This is the principal running mode. It has three slightly different ways of behaving and, depending on which way is desired, substitute A, B or C for the x. Basically, the only difference is in how certain conditions found in the envelope side are handled; the oscillator in all three modes just runs as usual.

To appreciate the subtle differences between these three modes, I present the following horror story: imagine that a generator is running, and the envelope part of the generator is happily running along cranking out ever louder values. Comes a time when this is supposed to stop, which means an update is supposed to arrive, to stop the increase, or something. But where's the update command?: stuck somewhere in the controlling computer which for some reason got too hung up to get the command out. The amplitude keeps growing until finally, we exceed the largest amplitude the generator can produce.¹ This can mean two things: first of all that the listeners have been blasted out of their senses by this out-of-control process, second, that as the amplitude continues to grow beyond its maximum value it wraps around to 0, suddenly superimposing silence from full amplitude, which is most likely a gross misrepresentation of the desired signal.

The three principal running modes differ basicly in how they would treat this situation. The envelope side of the generator can be made "sticky," which means that rather than overflow for lack of a command to tell it to stop, the envelope side of the generator will "stick" at the last value it attained before it would have overflowed. From that point on it will just continue to return that number until the update arrives to tell it to do something else. Alternately the envelope can be set to "free" mode, in which case the envelope wraps around and starts in at the beginning again, and all ones become all zeros and we start rising again (or all zeros become ones and we continue falling). A_RUNNING sets the envelope in sticky mode. B_RUNNING sets it in free mode.

¹ Arithmetic overflow occurs when a register is given a value that is greater than its capacity to represent, in either the positive or negative domain.

There's another mode whereby arithmetic overflow in an envelope can be detected and used as a "trigger" to cause the next higher numbered (the subsequent) generator to go from G_WAIT to running. Meanwhile the triggering generator goes to G_WAIT mode. This is what C_RUNNING does. The purpose of C_RUNNING is that by dedicating, for instance, three sequentially numbered generators to one "instrument", such that one generator computes the attack, the next the steady state, and the last the decay, the "instrument" can, once it is started, run without any further update commands. This ties up three generators, but lightens the command stream by two update ticks per invocation of that instrument. It may also be useful in cases like the one described above where the general system load is such that the box couldn't get a command in time to save a generator from command underrun. The user would still have a guarantee that at least the note once started would continue along a prescribed course.

9.5. DAC_WRITE

mode makes this oscillator be a port between the sum memory location addressed by the word in FM and a DAC addressed by the word in SWEEP (GO). This is how to get sounds out to a sound system. What happens to SWEEP? This is an instance where the function of the generator modifies what the parameters mean. When the generator is in DAC_WRITE mode, it is no longer a generator in the sense of producing samples. It goes into the special mode of taking information from the sum memory location whose address is in FM and passing it to the DAC whose location is in SWEEP. All other variables in this generator have no effect in this mode.

9.6. READ_DATA

mode makes the generator read data from the controlling computer's memory through the DMA channel, and deposit it in the sum memory addressed by SUM_MEMORY. This is how to get sound samples (for instance) into the box. The control logic in the synthesizer figures out how to get the data to the generator. The generator's only job is to plunk it down in sum memory.

9.7. sample interleaving for read/write data

It is possible to have all 256 generators in READ_DATA (or WRITE_DATA) mode. On every pass each generator will be given the next sample of data. Thus if you had a stereo sound file where the samples were interleaved (a,b,a,b,a,b...) then you would put two generators in READ_DATA mode. On one pass, two samples would be picked up from the input file data stream and sent (one each) to the generators. Since generators (as well as everything else) are processed in generator number order, the lowest numbered generator would be given sample a, and the higher would be given b.

9.8. WRITE_DATA

makes the generator take samples from a sum memory location and write them into main computer core memory through the DMA channel. Once in the controlling computer's memory they may be transferred to a permanent storage medium such as digital disk, tape, etc. Set up one generator in WRITE_DATA mode per channel of output. Again, the write data is interleaved in generator number order.

DATE!

DRAFT

DSK:GENDIA.XIP[DOC,MUS] GOES HERE

Section 10

Oscillator processing

In the diagram on page 24, the left side represents the oscillator processing, the right half the envelope processing. SWEEP (GO), RATE (GP) and ASYMPTOTE (GL) are constants which remain until altered by command, all other locations are "refreshing", meaning that their value is recomputed every time the generator goes through a processing tick. The old value is read by the process just below it, and the old value is added to the new value coming down from above. This is diagrammed as an arrow returning from the bottom to the top of such locations such as ANGLE (GK).

If an oscillator is in a wave producing mode, the following is a simplified description of what happens: (For the real version, see the [Specification]). On the oscillator side, FREQUENCY (GJ) will have the current frequency of the oscillator. SWEEP (GO) is a constant that is added into FREQUENCY (GJ) every pass to augment or diminish the frequency so as to produce frequency sweeping or glissando. (If you do not want any glissando, you must explicitly zero SWEEP.)

FM is the input port into the oscillator from the last pass sum memory. When the oscillator is in any oscillatory mode (as opposed to the data channel modes), it takes the sample from the sum memory addressed by the FM term, and the value obtained is summed with the value in FREQUENCY. Thus, the FREQUENCY can be modulated (changed) by the number in sum memory. So any processing element can affect the frequency of a generator by writing into the sum memory location that that generator reads through it's FM term. The value in FREQUENCY is not changed by the addition with FM, but a temporary location is reserved for the sum of these numbers which is not shown in the diagram.

Note that each generator, when in an oscillatory mode, always takes in a number through FM! If you don't want anything here, then you have to give FM the address of some sum memory location that is unused, so it will always read 0.

ANGLE (GK) stores the current phase angle. On every pass it is updated by adding the number in the temporary frequency location as computed above to the number in ANGLE (GK) left from the last pass. ANGLE is the index which keeps track of where we are in the cycle of whatever waveform we are producing. In general, ANGLE should be thought of as a signed binary ¹ counting register that, as it is incremented, represents the phase angle of one cycle of the waveform being generated. The phase angle of zero is represented by all zeros in the register. Pi/2 radians (90 degrees) is represented by 010000... (decimal +.5, i.e. one half of the positive range of 0 to +1-ε). Pi radians, or 180 degrees, is represented by 10000...(decimal -1, remember the high order bit is a sign bit!). And when ANGLE is all ones, the phase angle is about ready to start another cycle. When the angle value overflows, (exceeds all 1's) the amount of overflow appears as its new value, and we have started a new cycle of our wave at that angle. Thus the size of the number in FREQUENCY determines the frequency (oftenness) that ANGLE goes full circle.

¹ for a discussion of number representations used in this manual see Appendix 3.

When we finally have the value in `ANGLE`, what happens next depends on the mode of the oscillator. If, for instance, it is in sine wave producing mode, the `ANGLE (GK)` is used to lookup a value on a sine table. The value from the table is then the end result of the oscillator side. This result is not complete, however, as the output of the oscillator must still be scaled by the envelope side of the generator. The envelope side is discussed in the next section, and the rest of the modes are covered in more detail on page 33.

Section 11

Oscillator modes

These modes set the type of waveform the oscillator side of the generator will produce.

Definition	Function
SAWTOOTH	sawtooth wave
SQUARE	square wave
SUM_OF_COSINES	sum of cosines
PULSE_TRAIN	pulse train
SINE	sine(ANGLE), normal sine wave mode (with fm).
SIN_FM	cosine(FREQUENCY + FM) special sine table lookup mode.

Fig. 6

11.1. SINE

This mode is the standard sine wave oscillator. The value of the ANGLE term is used as an index into a sine table, the result of which is the end term of the oscillator. Actually only one quarter of the sine table is stored in the box. The other three quadrants of the full sine function have the same absolute value as the first, only sometimes retrograde and/or with a change of sign. This represents a considerable savings of memory. The ANGLE term is computed to 20 bits of precision, but only the high order 13 bits are used in the table lookup. The high order bit is taken as a sign bit, the next bit is taken as which quadrant, and the rest is used to access the actual table. Thus the table is only 2^{11} or 2048 locations long, instead of 8192, a nice savings. The precision of the sine table is 13 bits magnitude, plus sign bit = 14 bits total.

11.2. SAWTOOTH

For this, the oscillator merely has to pass on the value in ANGLE directly since the angle consists of a ramp function, which is another name for "sawtooth" waveform.

There is one problem, however. Remember that the ANGLE term is a signed two's complement number,¹ and remember also that this means it's largest positive value is $1-\epsilon$ whereas it's largest

¹ for a discussion of number representations used in this manual see appendix 3.

negative value is -1. This means that the waveform would be weighted in the negative domain by $-1/2$ of the least significant bit. This doesn't seem like much perhaps, but this is a D.C. offset in the signal nonetheless. The box avoids this by detecting when ANGLE is exactly -1 (binary 1000...) and substitutes the value of 0. So, here's the itinerary for a positive going ramp: ANGLE=RESULT: 0=0, .5=.5, $1-\epsilon=1-\epsilon$, -1=0, $-.5=-.5$, $-\epsilon=-\epsilon$, 0=0.

Please note that since there is this glitch when ANGLE = -1, this means that sawtooth waveform does not make a good ramp generator, since for a ramp, we really want to have that value of -1, instead of 0 as the result. Fortunately, there are other ways to generate ramps in the box. Modifiers may be used to do this.¹

11.3. SQUARE

The oscillator produces square waves by simply testing the sign of the value in ANGLE. While ANGLE is positive (between 0 and $1-\epsilon$), the oscillator produces +.5 (01000... signed two's complement binary). While ANGLE is negative, the oscillator puts out -.5 (11000...). Thus, during one half of the period of the ANGLE term's complete cycle, the oscillator will be producing -.5, the other half +.5.

11.4. PULSE_TRAIN

During a pass in which ANGLE wraps around, a pulse of +.5 (01000..., signed) is produced, otherwise 0 is produced. Thus, once per period we get a single non-zero value.

11.5. SIN_FM

The oscillator output is determined by the sum of FREQUENCY (GJ) and FM. Notice that we are taking the sine of the frequency term, not the angle term, as we would for a normal oscillatory mode. This is another case where terms are used for something other than that for which they are named. What this mode does basically is just look up whatever it finds in sum memory location FM in a sine table. The addition of the FREQUENCY term is peripheral. This mode, therefore doesn't really oscillate as such. Its principal use is looking up sine values for certain filtering applications.

¹ See the description of uniform noise mode in section 17.

11.6. SUM_OF_COSINES

This allows the addition of many equal amplitude cosine waves together in one generator, rather than having to dedicate several to the task. The result of the sum of many equal amplitude cosines where the frequencies are integer multiples is a band-limited pulse train. This means that the waveshape approaches that of a pulse train, but the frequency bandwidth is no wider than the difference of the frequency of the highest component minus the frequency of the lowest. The frequency of the highest component of a generator in pulse train mode is infinite, therefore the bandwidth is also infinite.

11.7. A note on band limiting

A waveform that has an infinite number of partials is not band-limited (seems reasonable, as tautologies go...). Since we are working in a sampled-data world, any partials that exceed in frequency $1/2$ the sampling rate are "folded-over" back into the area below that limit and are included in the total spectrum in strange and miraculous ways. The modes SQUARE, SAWTOOTH, and PULSE_TRAIN are all non-band limited under all conditions. The SUM_OF_COSINES and SINE mode are band limited but SUM_OF_COSINES can still cause foldover if its highest frequency component exceeds half the sampling rate. SINE can also cause foldover if its frequency exceeds half the sampling rate. PULSE_TRAIN always has terrible foldover and is not usually what you want to listen to, unless you dig foldover that is. Its basic purpose is to provide periodic triggering for other processes. SQUARE and SAWTOOTH are technically not band limited, but the amplitude of the partials falls off rapidly enough so that if the frequency is not too high, the higher partials are not audible.

Section 12

Envelope processing

On the envelope side, a 12 bit unsigned fractional binary number (decimal range = 0 to 1) is produced which is used to scale the output of the oscillator by multiplication. This envelope value can be made to ramp up or down either linearly or exponentially by the hardware. Since the result of the oscillator side is signed, the multiplication is two quadrant.

12.1. EXPONENT and RATE

EXPONENT (GQ) stores the current value of the envelope ramping value. RATE (GP) is a constant which is added into EXPONENT every pass to increment or decrement it thus augmenting or diminishing the amplitude. Taken through time therefore, EXPONENT can be made to produce a linear ramp function from 0 to 1 (unless RATE is exactly 0, in which case EXPONENT remains constant).

This linearly ramping value in EXPONENT can either be taken directly, or it can be used as an index into a table of exponentials to produce envelopes which change exponentially. The method of lookup can produce either positive or negative exponential sections.

If the generator running mode is in one of the "sticky" modes, namely A_RUNNING or C_RUNNING, then the EXPONENT will not wrap around. It will "stick" at the last value attained before overflow. Note that this may or may not be the largest number it can represent. For instance, if RATE is a large number, then the nearest value EXPONENT could get to its maximum value before overflowing could be quite far.

12.2. ASYMPTOTE

The value in ASYMPTOTE (GL) is added to (or subtracted from) the value in EXPONENT and this final number is the one used to scale the output of the oscillator. The effect of ASYMPTOTE therefore is to offset the the value in EXPONENT by some constant value.

It can be used in conjunction with sticky mode as follows: Say you are producing an amplitude function and you want to start this segment at .16 and let it increment by some value for a few passes to .95. Well, let's remember the potential for disaster that can happen if the command that's supposed to catch it at .95 doesn't arrive in time. It would appear offhand that, since the value we want to stop at is well below overflow, we don't get to use the sticky envelope feature. But! Let's put an initial value of -.05 in ASYMPTOTE, and lets put $.16 - (-.05) = .21$ in EXPONENT. The number in EXPONENT now goes from .21 to overflow and will stick (assuming you've put the generator in the correct mode), while the actual output of the envelope side goes from .16 to .95.

If an exponential envelope curve is required, then EXPONENT (GQ) is used to look up a value in a table of exponents which is then returned instead. The linear value used to do the lookup is kept separate so as to be able to compute the subsequent table addresses. The value of ASYMPTOTE is added to (or subtracted from, depending on mode) the value from the exponential table.

The result of the envelope side is 12 bits, unsigned.

The result of the envelope process is multiplied by the result of the oscillator process. The size of the multiply is 13 bits from the oscillator (which is sign bit followed by 12 bits of magnitude) times 12 bits unsigned from the envelope. This constitutes a 2 quadrant multiply. The full precision result of the multiply would be $13+12=25$ bits, but only the high order 19 bits are taken. This is then loaded right adjusted in sum memory. This final result is added to whatever is the current value in the this pass sum memory location that the generator addresses. The reason the value is right adjusted in 20 bits is reputedly to provide a little de facto amplitude scaling, so that more than one generator can write into the same sum memory location at maximum amplitude without overflowing. This causes some complications for doing FM and waveshaping. For more on this subject see section 15.

Section 13

Envelope modes

These modes specify the algorithms for envelope processing.

Definition Function

LPLUSQ	Add EXPONENT (GQ) to the offset constant in ASYMPTOTE (GL).
LMINUSQ	Subtract EXPONENT (GQ) from offset constant in ASYMPTOTE (GL).
LEXPLUS	Scale $2^{-\text{EXPONENT}}$ and add to ASYMPTOTE.
LEXMINUS	Scale $2^{-\text{EXPONENT}}$ and then subtract from ASYMPTOTE.

Fig. 7

LPLUSQ: The value in EXPONENT is added to the constant in ASYMPTOTE.

LMINUSQ: Same as LPLUSQ, but subtract.

LEXPLUS: The value in EXPONENT is used as a lookup in an exponential table and the result is added to the constant in ASYMPTOTE. The exponent returned from the table lookup can be defined as $2^{-\text{EXPONENT} \times 16}$. To see what's happening here, first remember that EXPONENT takes values between 0 and 1. If EXPONENT is 0 then the value from the table will be 1 ($2^0 = 1$). If EXPONENT is 1/16, then the result is .5. If EXPONENT is 1, the result is 2^{-16} , taken as zero. Inbetween values are exponential. We could also look at it in binary. The following is all unsigned.

EXPONENT:		Result from table:	
binary	decimal	binary	decimal
0000 0000...	0	111 111 111 101	1
0001 0000...	1/16	011 111 111 101	1/2
0010 0000...	1/8	001 111 111 101	1/4
0011 0000...		001 111 111 101	1/8
0100 0000...	1/4	000 111 111 101	1/16
:			
:			
:			
1111 1111...	1	000 000 000 000...	0

So in the binary representation, it is like the EXPONENT has a 4 bit integer part (the high order bits) and an 8 bit fractional part (the rest of the word).

LEXMINUS: Exponential table lookup is done, and value is subtracted from the constant.

Section 14

More on generator processing - alternate modes

As you know, the generator parameters must do double duty sometimes depending on whether they are acting as data paths or waveform generators. Here we get into the guts of the issue.¹

The generators function in a variety of ways in the box. Their complete list of responsibilities are: 1) waveform production, 2) the transfer of data from sum memory to CPU, 3) the transfer of data from CPU to sum memory, 4) the transfer of data from sum memory to the DACs and 5) doing sine table lookups on data in sum memory. All of this is done by making various generator parameters function in more than one role. Each generator running mode causes different generator fields to behave in different ways, so now we need a "road map" which gives a close (though not definitive) look at them.

Remember that the generators have two parts, an oscillator part and an envelope part. The oscillator side of the generator is running if it is in any of G_WAIT, A_RUNNING, B_RUNNING C_RUNNING and READ_DATA modes. It is not running in INACTIVE, G_PAUSE, WRITE_DATA and WRITE_DAC modes. By "running" I technically mean whether ANGLE is being updated or not. A special exception to this rule is SIN_FM where part of the oscillator is running (the sine table lookup part) but the rest of it is not.

So here is a list of the generator parameters, and a description of what part they play in the various run modes.

14.1. SWEEP

SWEEP (GO) is the oscillator frequency sweep rate. SWEEP stores a number which, when the oscillator is running, is added to FREQUENCY (GJ) on every pass. This allows FREQUENCY to rise or fall at a given rate. SWEEP functions as such in all oscillator run modes. If the generator is in WRITE_DAC mode, then SWEEP is used to store the address of the DAC, and its value is not added into FREQUENCY.

14.2. FREQUENCY

FREQUENCY (GJ) stores the oscillator frequency. The data from sum memory addressed by FM is added to the value in FREQUENCY in all oscillator modes. FM will presumably address

¹ Note, this is not the order of the processing steps. For the definitive processing order, as well as all the gory details, see the [Specification].

either frequency modulation data from some other processing element, or will have data to be sent to a DAC. If the oscillator side is running then SWEEP is also added to FREQUENCY (as described above, this increments the frequency by a constant, for frequency slewing, or glissando).

14.3. ANGLE

ANGLE (GK). The primary function of ANGLE is to accumulate the instantaneous index which represents the current phase angle of the waveform being generated. This term is used as an index to lookup sine values on a table or as the datum of a logic test for the straight line waveforms. However, it is used this way only when the oscillator is running.

The principal exception to this is SIN_FM. In SIN_FM mode, ANGLE is not used at all. Instead, the output of the oscillator is the sine of the sum of FREQUENCY + <value addressed by FM>.

If the generator run mode is WRITE_DATA or WRITE_DAC then the data sent to the CPU is the word addressed by FM.

Now a word for the cognoscenti (the rest of you can skip this paragraph). Note from the [Specification] that in modes SINE and SUM_OF_COSINES there is an implicit constant phase shift of $2\pi \cdot 2^{-13}$ added to the angle term before lookup. The entire table is shifted "one half" a position to the right, as it were. The problem comes in implementing the SUM_OF_COSINES mode. In that mode the first step is to take the CSC of the ANGLE term. The CSC function is $1/\text{SIN}$, and if we took the sin of an angle of 0 we'd get infinity. But by shifting the ANGLE term in this way we only get "nearly" infinity, so it works out. This shift is basically transparent to everything one would use the box for, it won't screw you unless you try to do Andy Moorer's sine summation synthesis, or something, so relax.

14.4. NCOSINES and SCALE

N_COSINES (GN), SCALE (GM). These are relevant only for SUM_OF_COSINES mode. N_COSINES specifies the number of cosines to be summed. SCALE gives a scale factor for normalizing the summed result. (If you have a bunch of cosine waves, remembering that they all have an amplitude of 1 at zero degrees, their sum can begin to stack up, hence the need for normalization.) Actually, the SUM_OF_COSINES algorithm should be normalized by $1/\text{NCOSINES}$, but we don't have a divider at that point in the processor, so we approximate it by just scaling the result of the cosine summation formula by shifting the bits in the word right or left, which you recall is the same as dividing or multiplying by a power of 2. This gets us within a factor of 2 of the correct normalization, not perfect, but acceptable.

Please note that SCALE should be set to zero when the oscillator mode is not

SUM_OF_COSINES ¹ Note that this also means that in SUM_OF_COSINES mode, SCALE should be non-zero.

All of this notwithstanding, please also see the reference in section 15 for other uses of the SCALE term.

14.5. FM

FM (GFM) has the address of a location in sum memory which the generator can read. FM can read from either generator or modifier last pass. FM is read and its value is added together with FREQUENCY in all oscillator modes. It reads in all 20 bits from sum memory. Please note that if you don't want data from sum memory then you must give FM the address of some location in sum memory which is never written into and will hence always be zero.

14.6. SUM_MEMORY

SUM_MEMORY (GSUM) has the address of a location in sum memory into which the generator writes its output. This output is generally added to the contents already there from this pass. But if the run mode is READ_DATA, then sum memory gets the final output of the generator PLUS the data read from the main computer CPU. This REPLACES the contents of the sum memory location addressed (so it's no longer really sum memory).

14.7. RATE EXPONENT and ASYMPTOTE

RATE (GP), EXPONENT (GQ) and ASYMPTOTE (GL) are relevant only when the oscillator is in a wave producing mode and the envelope is running as well. The envelope side is running in A_RUNNING, B_RUNNING and C_RUNNING only. EXPONENT is the envelope increment corresponding in function to FREQUENCY in the oscillator. It can be augmented or diminished by a number in RATE just as FREQUENCY can be changed by SWEEP. EXPONENT is either used directly as a linear function or as an index to an exponential table used to produce exponential envelopes. The resulting number returned from either path is added to ASYMPTOTE, which is a constant used to offset the envelope ramp from the X axis.

¹ This is because a non-zero value in SCALE implies a subtraction of one. This is necessary for the SUM_OF_COSINES algorithm to get rid of the cos(theta) term. For a discussion of sine summation synthesis see [Winham].

Section 15

Generator magic numbers

This section shows how to translate from frequencies specified in Hz to the appropriate values for the Synthesizer. In addition, some other inner details of the generators is covered, including solutions to some problems of implementing FM.

15.1. Specifying frequencies to generators

As we've seen, all processing that results in output is in terms of passes. We know that the EXPONENT term is ramped up or down by a fixed amount specified in RATE on every pass. So the value in EXPONENT after a certain number of passes equals the starting value in EXPONENT, plus the value in RATE times the number of elapsed passes. Thus the envelope increases or decreases depending on the value in RATE and the number of passes. The same is true of FREQUENCY and SWEEP. In fact, the same is true of ANGLE and FREQUENCY. The ANGLE equals the value in FREQUENCY times the number of passes elapsed. In other words, the higher the incremental value in FREQUENCY, the more frequently ANGLE goes through its cycle.

To figure out just how to set FREQUENCY to get a particular pitch depends on knowing the sampling rate and some details of the FREQUENCY term. To be really clear, let's take some examples first.

You remember the unit circle from trigonometry, right? Well, as ANGLE goes from 0000... to 1111... we go through a complete cycle of our waveform. Now, think about the range of frequencies available. When FREQUENCY is 0, ANGLE is not incremented, and we have zero oscillation. When frequency is set to +.5 (signed binary 01000...) it will cause the ANGLE term to wrap around every other pass, which means we are producing an oscillation at 1/2 the sampling rate. If FREQUENCY is set to its maximum positive value of 1- ϵ , ANGLE will wrap around on nearly every pass. Thus the frequency produced when FREQUENCY is nearly one is nearly equal to the sampling rate.

So you can see that the frequency of a generator is a function of the speed at which FREQUENCY makes ANGLE go through a complete cycle, and this in turn is controlled by how many passes are executed (and therefore samples produced) per second.

We also see that the range of actual frequencies we can specify with the FREQUENCY field is between 0 Hz and a little less than the frequency of the sampling rate. So we can think of the FREQUENCY term as a fraction which specifies where in that range we are. So we want to develop a formula to express this. As the FREQUENCY fraction goes to its maximum positive value, 1, the frequency goes to the sampling rate. So we have the basic identity ratio of $1 =$

maximum_frequency/sampling_rate. This magic ratio is used so often that it is given a special name: MAG. Now to get a particular frequency, we must express it as:

$$\frac{\text{desired_frequency(Hz)}}{\text{maximum_frequency(Hz)}} * \frac{\text{maximum_frequency(Hz)}}{\text{samples/sec.}} = \text{FREQUENCY} \quad \text{Equ. 7}$$

One last detail. The value for maximum_frequency in the Samson box is 1-ε in the FREQUENCY field, which corresponds to signed binary .011111... Since it is a 28 bit field, that means maximum_frequency is 2^{28} . The value of the sampling rate varies, depending on the speed of processing. So the constant is always recalculated whenever the sampling rate changes.

For a concrete example, say we had a sampling rate of 12800 samples per second. MAG would then be $2^{28}/12800 = 2^{28}/(2^7*100) = 2^{21}/100 = 20971.52$. This number multiplied by some desired frequency will put the correct binary number in FREQUENCY. For instance, 1/4 the sampling rate = 3200 Hz = 2^5*100 .

$$2^5*100 * \frac{2^{21}}{100} = 2^{26}, \quad \text{Equ. 8}$$

which in a 28 bit signed fractional binary word is .010000...

15.2. More about the FREQUENCY parameter

The low order eight bits (the rightmost bits) of FREQUENCY are not added into ANGLE at all, rather, the high order 20 bits of FREQUENCY are all that are added to ANGLE. So what good are the low order 8 bits? The point is that we want to try to get as fine a change of frequency as possible when FREQUENCY is being incremented by SWEEP so that we can represent extremely slow changes in frequency. To this end, SWEEP is added into FREQUENCY right adjusted, meaning its 20 bits are added to the righthand, or low order, 20 bits of FREQUENCY's total of 28 bits. But since only the high order 20 bits of FREQUENCY are used to add to ANGLE, the right eight bits act only as an accumulator of small changes of frequency. When the eight bits finally sum up enough to toggle the 9th bit, then the amount is registered as part of the new 20 bit frequency increment sent to ANGLE. Thus we get eight more bits of frequency precision for small frequency changes. These extra 8 bits are best thought of as a fraction tacked on to the FREQUENCY term.

15.3. SWEEP scaling

Another useful formula is for SWEEP, the frequency sweep rate. To get a frequency sweep from Frequency_1 to Frequency_2 in one second, put Frequency_1 times the box's magic number in FREQUENCY (GJ), and the difference of Frequency_2 - Frequency_1 times another magic number, $2^{28}/\text{clock_rate}^2$, in SWEEP. That would be:

$$\text{FREQUENCY} \leftarrow \text{Frequency}_1 * 2^{28} / \text{clock_rate},$$

Equ. 9

and

$$\text{SWEEP} \leftarrow (\text{Frequency}_2 - \text{Frequency}_1) * 2^{28} / \text{clock_rate}^2.$$

Equ. 10

The derivation of the formula for SWEEP is as follows:

$$\text{SWEEP} \leftarrow \frac{(\text{Frequency}_2 * 2^{28} / \text{clock_rate}) - (\text{Frequency}_1 * 2^{28} / \text{clock_rate})}{\text{clock_rate}},$$

$$\text{SWEEP} \leftarrow \frac{(\text{Frequency}_2 - \text{Frequency}_1) * 2^{28} / \text{clock_rate}}{\text{clock_rate}},$$

$$\text{SWEEP} \leftarrow \frac{(\text{Frequency}_2 - \text{Frequency}_1) * 2^{28} * 1 / \text{clock_rate}}{\text{clock_rate}},$$

$$\text{SWEEP} \leftarrow (\text{Frequency}_2 - \text{Frequency}_1) * 2^{28} * 1 / \text{clock_rate} * 1 / \text{clock_rate},$$

$$\text{SWEEP} \leftarrow (\text{Frequency}_2 - \text{Frequency}_1) * 2^{28} / \text{clock_rate}^2.$$

Equ. 11

To make this sweep happen in some other than one second, simply divide the above number by the desired duration.

15.4. FM scaling

For doing FM in the box, the generators take the ANGLE term as the sum of the FREQUENCY term and the value in sum memory pointed to by the FM address. This section tries to get a handle on the actual effect of frequency modulation in the box using generators as the source of modulation.

The width of ANGLE is 20 bits. This means that the highest frequency representable is one which turns ANGLE around once each pass. The number which would do this is 2^{20} , which if looked at in one way can be thought of as a 1 stuck out in the place one beyond the 20th bit

position, followed by 20 zeros. There can't really be a bit out there, but if there were, it would represent the units place, and the rest of the bits to the right would be the fraction. A frequency that goes through one cycle every pass is going to be equal to the frequency of the sampling rate. Call the sampling rate F_s . Since this hypothetical 1 in the 21st bit position is off the end of the 20 bit table followed by all zeros, it is also identical to a frequency of zero Hz, because the actual bits that make up the ANGLE term in both cases are all 0's. This corresponds to the fact that the radian frequency of 2π and its multiples are equal to the radian frequency of 0.

So we can say that there is a correspondence between F_s and the width of the ANGLE term, 2^{20} .

Note that the width of sum memory is also 2^{20} . If we were to put a 1 in the low order bit of sum memory addressed by a generator, it would raise the value in the ANGLE term to 1 more than would have been the result of adding in the FREQUENCY term alone, hence we have modulated the frequency.

Lets calculate the actual effect this has on the frequency of the carrier. First, what is the maximum modulation? Remember from the last section that maximum positive frequency is $1-\epsilon = 01111\dots$. Since the sum memory term is 20 bits, and it is added left adjusted into the ANGLE term, this is represented as $2^{20}-1$. So the frequency of a generator can be altered to be anywhere between 0 Hz and nearly the sampling rate.

What is the minimum step size of the modulation? Well, a 1 in the least significant bit would be the minimum positive frequency that is still greater than 0 Hz. This would be represented as $000\dots001 = 2^0$. Since the maximum modulation is $F_s/2^{20}$, the minimum value would be expressed as a fraction of this, or $2^0 * F_s/2^{20}$. We can say that the modulation is in terms of unnormalized multiples of $F_s/2^{20}$. For a concrete example, take a sampling rate of $F_s=37\text{kHz}$. The smallest modulation increment is then $1*37\text{kHz}/2^{20}$, or about .035 Hz. This value is less than a cent down to the range of around 100 Hz, meaning that there will be no perceptable jumps in frequency.

There is a complication in implementing FM using the box's generators as modulators, however. Remember that the result of a generator is a 19 bit product, shifted right one place in sum memory, sign extended, making an absolute maximum magnitude of 2^{18} . That means the greatest number that can be added to another generator is 2^{18} , which equals $F_s/4$. So one generator can produce an absolute maximum deviation of not quite one quarter of the sampling rate. Deviations greater than this are sometimes encountered in practice, and it is an unfortunate necessity to have use a modifier to scale the output of the modulating generator up by the appropriate amount to achieve the correct depth of modulation.

There's also a problem in generating small modulation depths. Such small modulation depths are used often for getting vibrato effects, where the pitch does not want change by much. The minimum non-zero output of generators is where the envelope term is its smallest non-zero value, which is 1 in its LSB. The result of this is, again, the high order 19 bits of the product, right adjusted in sum memory, sign extended. If you figure it out it means that the minimum positive amplitude that ends up in sum memory is $2^{7-1}-1$, which is considerably greater than the minimum frequency modulation resolution of the carrier which reads this value. The perceptual result is either a large amount of vibrato, or, below a certain threshold, none at all. Two things can be done about this: 1) use a modifier to scale the modulation signal down to the correct range, or 2)

kludge it by increasing the SCALE term in the modulating generator. This shifts the oscillator product right before multiplying with the envelope part, and you get a usefully small modulation depth.

To calculate the frequency deviation for a particular modulation index and modulating frequency, would be:

$$|\text{deviation}| = \text{index} * \text{modulating_frequency} / \text{maximum_deviation} \quad \text{Equ. 12 .}$$

In practice because of the above-mentioned quirks of the box's generators, the maximum_deviation term is equal to the sampling_rate/8.

Section 16

Modifiers

16.1. Modifier parameters

Modifiers are so named because they are mostly used to modify existing signals, by filtering, reverberation, amplitude scaling, etc. To do this they are provided with two input ports from sum memory and one output port back to sum memory. They can do two multiplies, which is why they take two ticks, whereas generators, which do only one multiply take only one tick. The whole list of their parameters or fields is as follows. All these fields can be loaded on update ticks.

Id	Size	Definition	Function
M0	30	COEFF0	coefficient
M1	30	COEFF1	other coefficient
L0	20	TERM_0	running term
L1	20	TERM_1	other running term
MIN	8	A_IN	read address in sum memory
MRM	8	B_IN	other read address
MSUM	7	SUM_MEMORY	write address in sum memory
		ADD_SUM_MEMORY	same as SUM_MEMORY.
		REPLACE_SUM_MEMORY	also has write addr. but result replaces sum mem. value
MMODE	9	MODE	run mode of modifier with A_SCALE and B_SCALE.
		A_SCALE	see section 16.
		B_SCALE	see section 16.
FUNCTION			same as MODE, but without specifying the SCALE terms.

Fig. 8

16.2. Read terms

A_IN and B_IN are terms which contain the address of a location in sum memory used to fetch samples from sum memory.

16.3. Coefficient terms

COEFF0 AND COEFF1 are the constants in the two multiplies that the modifier does. Depending on the mode, they are used to scale either the running terms or the read terms. A couple of the filtering modes have provisions to ramp the coefficients, but otherwise they are constants.

16.4. Scale terms

The product of the multiplies in a modifier can be scaled up, and sometimes down by a constant power of 2, stored in A_SCALE and B_SCALE. For basic information about the effect of the scale terms see below under Modifier Arithmetic.

The results of the values of the scale terms is as follows:

Scale value	multiply is scaled by:	
	fraction:	integer:
00:	x 1	x 1/4
01:	x 2	x 1/2
10:	x 4	x 1
11:	x 8	x 2

Fig. 9

Notice that the scale commands are a part of the MODE command. It worked out that there was enough room left in the MODE command to accommodate this extra baggage instead of having to have an extra class of command specifically for this.

The last detail to mention is which scale term affects which multiply: First, the order of the multiplies in the hardware is always fixed: a multiplication involving COEFF1 will be the first multiplication, one involving COEFF0 will be the second. (Seems backwards, doesn't it?) Now, the scale term used for the first multiplication is B_SCALE, the term for the second is A_SCALE, so we get the pairing: COEFF1_{B_SCALE}, COEFF0_{A_SCALE}.

16.5. Running terms

There are occasions when a modifier needs to keep data from one pass to the next, such as in the filtering modes and certain others. TERM_0 and TERM_1 are registers in the modifiers which will have as their contents whatever was last stuffed into them, even across passes.

16.6. Mode and Function

The function of the modifier states what algorithm it will execute. The Mode is broader, and includes the specification of the scale terms. Function is therefore only that part of the command which selects the algorithm, Mode is the entire command. For a complete account of the various flavors, see section 17.

16.7. Sum memory

This determines where the modifier will write in the modifier last pass quadrant of sum memory. Modifiers have an option here that generators do not: besides just summing with the contents of the addressed sum memory location, modifiers can also replace the value in sum memory with their value. There are three mnemonics for this: `SUM_MEMORY` is the same as `ADD_SUM_MEMORY` and for both, the result of the modifier procedure is added to the preexisting contents (if any) in the sum memory location addressed. To replace the contents of sum memory say `REPLACE_SUM_MEMORY`.

16.8. Modifier arithmetic

The minimum data path through all parts of the modifiers is 20 bits. The two coefficients are 30 bits, but only the high order 20 bits are used in the multiplies. The remaining 10 bits are used in some of the filtering modes where the coefficients are incremented internally, and these extra bits are to add precision to slow moving changes. The reasoning is the same for the low order 8 bits of `FREQUENCY` in generators.

Of the 40 bit product which results from the 20*20 bit multiplies, either the high or low order 20 bits is returned depending on whether the modifier mode uses fractional or integer arithmetic. Most modes use fractional arithmetic, but a couple are in integer: integer mixing, uniform noise and triggered uniform noise. For these, the low order 20 bits of the multiply are returned, and any overflow is ignored.

The arithmetic difference between integer and fractional multiplies is as follows: In fractional notation, all numbers are considered to be in the signed unit interval, that is the range between (decimal) .999... and -1.000...0. So the product of a fractional multiplication will never be greater than either of the coefficients. For example, in decimal, $.9*.9=.81$, which is smaller than .9.

Now in the integer case, if, for instance we multiply two 5 bit binary integers, 00010. times 00010., we get 00100. (decimal $2*2=4$) and the product will always be greater than or equal to the largest coefficient. If we used larger and larger values for coefficients we would get a result that is larger and larger: $00100*00100 = 10000$, etc. Eventually the product would overflow off the left of the number of bits we have to represent the number. In this case (taking the width of the words in the example as 5 bits), 10000 times anything greater than 00001 will overflow beyond the 5th bit

position. This is usually a gross error since the difference between what we have in the register and the true product can be off by the entire word length. In the modifier modes that use integer multiplies, this overflow is ignored (although the perceptual result wouldn't necessarily be ignorable!).

The SCALE terms can be used to shift this product right and sometimes left to make it bigger or smaller. It can be easily seen that shifting binary 10 left one place, making it 100 is the same as taking 2^1 and making it 2^2 , i.e., multiplying by a power of two for every position shifted. Similarly shifting a binary word right n places is the same as dividing by 2^n . Fractional modifier modes always treat the scale terms as left shifts (see the table above under A_SCALE, B_SCALE) while integer multiplies have provisions for going either way. More particularly, fractional multiplies can be shifted left 0, 1, 2 or 3 places. Integer multiplies can be shifted right 1 or 2 places and shifted left 0 or 1 places.

Some implications of all of this:

Notice that when integer mixing is scaled right, the sign bit must be copied back into the space vacated at the top of the word, which is called extending the sign. There is no way to avoid having the sign extended. Indeed, in fractional multiplies the sign is also always extended (since the value is always smaller than the original quotients, before any shifting).¹

You can't get a multiply by exactly integer 1 in a modifier mode that uses fractional arithmetic because the largest positive value is $1-\epsilon$. If that's close enough for you, fine. But if you want exactly times 1, you can get it by setting the coefficient of the multiply in question to .5 and setting its corresponding scale term to shift the result left one place. Even this is not without side-effects, however. The result of the fractional multiply, you remember, is the high order 20 bits of the 40 bit product. When this is shifted left by the scaling operation specified in the SCALE bits, it does not start shifting in bits from the low order 20 bits, but brings in 0's instead. This means you lose a small bit of precision. This is usually infinitesimal, but a point worth bearing in mind. Of course it's trivial to multiply by exactly 1 in integer arithmetic. If you are just copying signals from one pass to the next, this is the way to do it.

16.9. Modifier this pass timing considerations

This section can safely be skipped by just about everyone.

Modifiers have the ability to read this pass sum memory. The reason for the division of this pass and last pass in general is that one is guaranteed that all processes have generated their output. Reading things from this pass is more difficult because one must make sure that the process one is reading has occurred long enough ago in this pass period to have produced its output. Here then is the information necessary to make this determination.

¹ There can be times when this is a problem. However, by using a modifier in MAXIMUM mode, reading in the signal on both A_IN and B_IN, and multiplying one of them by $-\epsilon$, the effective result is to remove the sign bit.

Modifier number M writes into sum memory (either summing or replacing the contents) on tick number $2*M + 7$.

Modifier number M reads its B_IN port on tick number $2*M$.

Modifier number M reads its A_IN port on tick number $2*M$ in modes: `INT_MIXING`, `MIXING`, `SIGNUM`, `MINIMUM`, `MAXIMUM`, `AM`, `FOUR_QUAD_MULTIPLY` and `THRESHOLD`.

Modifier number M reads its A_IN port on tick number $2*M + 6$ in modes: `LATCH`, `ONE_ZERO`, `INVOKE_DELAY_UNIT` and all six forms of `two_pole` and `two_zero` modes.

Section 17

Modifier modes

The name of the process is followed by its reserved definition followed in turn by an explanation. Unless otherwise stated, the computation is in 20 bit two's compliment, fixed point fractional notation.

17.1. Inactive

M_INACTIVE

Dead to the world. As with generators, being inactive does not clear its fields. The parameters (COEFF0 and COEFF1 and such) will be left at whatever they were before the modifier went inactive. It will also still occupy its place in the ordered processing of modifiers and use up two ticks.

17.2. Mixing

MIXING

$\text{Result} \leftarrow \text{COEFF0} * \text{A_IN} + \text{COEFF1} * \text{B_IN}.$

Signals read from A_IN and B_IN are multiplied by COEFF0 and COEFF1 respectively and added together. The COEFF terms act here and in many other modes as gain factors for the weighting of the respective input terms. The result is placed back in sum memory location referenced by MSUM. A_IN and B_IN will usually be used to lookup sum memory locations which will usually contain current instantaneous values of waveforms being created by other processing elements. The multiplications are in fractional form as described above.

17.3. Integer mixing

INT_MIXING

$\text{Result}(\text{integer}) \leftarrow \text{COEFF0} * \text{A_IN} + \text{COEFF1} * \text{B_IN}.$

Computation is the same as for fractional mixing above except that integer multiplying is done.

17.4. Amplitude modulation

AM

$\text{Result} \leftarrow \text{TERM}_1 * \text{COEFF1}$. $\text{TERM}_1 \leftarrow \text{A_IN} * ((\text{B_IN} + 1) / 2)$.

This multiplies two waveforms together such that B_IN is used to scale the amplitude of A_IN. This is done by multiplying the number to be scaled by another which has values between 0 and 1. If this scaling is done through time, the overall result is of the amplitude of A_IN being dynamically modulated by B_IN. To do this, we first need to scrunch B_IN so its value lies between 0 and 1. But B_IN is a binary point number which has values between .999... and -1.000... So to convert B_IN, first add 1 to B_IN, to make its range go from 0 to 2. Then divide that by 2 to make it go between 0 and 1. Now we multiply A_IN*B_IN and return the product. This process is repeated for every point on the waveforms A_IN and B_IN. The steps the box takes to do this are as follows. The value TERM_1*COEFF1 is returned where TERM_1 was determined the last time around to be A_IN*((B_IN+1)/2). This process constitutes a two quadrant multiply in that the sign of A_IN is significant but that of B_IN is not (B_IN, being scaled between 0 and 1 will always be positive). So, if we plotted the point (A_IN,B_IN) on a graph, any product of A_IN and B_IN would only need two quadrants to be represented.

17.5. Four quadrant multiply

FOUR_QUAD_MULTIPLY

$\text{Result} \leftarrow \text{TERM}_1 * \text{COEFF1}$. $\text{TERM}_1 \leftarrow \text{A_IN} * \text{B_IN}$.

TERM_1 scaled by COEFF1 is returned. A new TERM_1 is then computed as A_IN*B_IN and stored in TERM_1 to be multiplied by COEFF1 next time around. Here, both the signs of A_IN and B_IN are significant. If we were to plot the point (A_IN,B_IN) on a graph, the point could be in any of the four Cartesian quadrants, which is where the "four quadrant" comes from. For analog synthesizer folks, this is the ultimate ring modulator.

17.6. Latch

LATCH

$\text{Result} \leftarrow \text{TERM}_1$. If $\text{COEFF1} * \text{B_IN} \neq 0$ then $\text{TERM}_1 \leftarrow \text{A_IN}$.

The modifier gives as its result whatever was in TERM_1 from last pass, then it looks to see if it should read in a new value from its A_IN address. It tests to see if COEFF1*B_IN≠0, and if so it looks up the value of A_IN and replaces TERM_1 with it. Otherwise, TERM_1 hangs around until next time. The purpose of this function is to wait for a triggering condition to be true, then sample a changing variable being stored in sum memory (like to look at a sine wave being fed

into sum memory) and then to return the value sampled. If the condition becomes not true then the value of the last sample taken before the condition became false is returned. This value will be returned repeatedly until the condition is true again at which time a new sample will be taken and returned. (Since a running term is used, it is necessary to clear it first to get predictable behavior.)

17.7. Signum

SIGNUM

If $\text{COEFF0} * \text{A_IN} < \text{COEFF1} * \text{B_IN}$ then the result $\leftarrow -1$ (integer) (= all bits set)
else if the products are equal then the result $\leftarrow 0$
else if $\text{COEFF0} * \text{A_IN} > \text{COEFF1} * \text{B_IN}$ then the result $\leftarrow 1$ (integer).

This tests the sign of the expression. It is similar to the Fortran "numerical if" statement in that it allows one of three values to be returned as the result of an expression being $>$, $=$, or < 0 . The result is in integer form; the multiplies are fractional.

17.8. Minimum

MINIMUM The lesser of $\text{A_IN} * \text{COEFF0}$ or $\text{B_IN} * \text{COEFF1}$ is returned.

17.9. Maximum

MAXIMUM The greater of $\text{A_IN} * \text{COEFF0}$ or $\text{B_IN} * \text{COEFF1}$ is returned.

17.10. Threshold

THRESHOLD

If $\text{COEFF0} * \text{A_IN} + \text{TERM}_0 < 0$ then result $\leftarrow 0$ else if ≥ 0 then result $\leftarrow \text{COEFF1} * \text{B_IN}$.

This allows one to detect when a signal crosses some amplitude threshold. The tested value is $\text{A_IN} * \text{COEFF0} + \text{TERM}_0$. If it is less than 0 then 0 is returned, else if it is greater than or equal to 0 then $\text{B_IN} * \text{COEFF1}$ is returned. Thus the effect is like a valve, letting $\text{B_IN} * \text{COEFF1}$ through when the tested expression is true. So B_IN could pass some value acting as a trigger to some other process to turn it on or off, etc.

17.11. Zero-crossing pulser

ZERO_CROSSING_PULSER

If $TERM_1 * COEFF1 \neq 0$ and either $B_IN * COEFF0 = 0$
 or if $(B_IN * COEFF0) * (TERM_1 * COEFF1) \leq 0$ then result $\leftarrow -\epsilon$
 (all bits on) otherwise result $\leftarrow 0$.
 $TERM_1 \leftarrow B_IN * COEFF0$.

This one figures out waveform zero crossings. First let's consider when a waveform can be considered to have crossed zero. There are two cases to consider. Look at two consecutive samples.

Case 1) If the first one is not zero and the second one is, then we've arrived at zero (which is taken as crossing it).

Case 2) If one of them is negative and the other positive, we consider that we've crossed zero. This is tested by taking the product of the two points. The result of multiplies of mixed sign is always negative, i.e., less than zero, so we test if the product is less than zero.

There is also a third case: We must test for the case where both samples are zero. We don't want to consider this a zero crossing, because nothing has been crossed. If the zero crossing being detected is so gradual that it includes several consecutive zero samples in the middle, then we'll just have to settle for having the crossing detected at the start of the crossing, rather than at its hypothetical mid-point.

The modifier routine will do just exactly what I've described, take a close look.

If a zero crossing is detected, the number $-\epsilon$ is returned, otherwise 0. The term $-\epsilon$ is the smallest negative number, which in our case is represented as the binary number with all bits set.

17.12. Uniform noise

U_NOISE

$TERM_1 \leftarrow Result \leftarrow COEFF0 * TERM_1 + TERM_0$.

To produce noise and unpredictable variation in a digital signal requires that a source of random samples is available. For this purpose a "psuedo-random" number generator is used. It is "psuedo" in the sense that it is not really random, but has some (hopefully large) periodicity.

One type of random number generator is called a *linear congruential generator* and the U_NOISE modifier mode is provided to implement it on the Samson box. In U_NOISE mode, $TERM_0$ is added to the integer product of $TERM_1 * COEFF0$ and the result is returned. Overflow is purposefully ignored here, since this is where the randomness comes in! This means the *low order*

20 bits of the product are used. It overwrites TERM_1 with this result, to be used next time around.

To produce good uniform or "white" noise the numbers used in the procedure must be chosen with care. The theory behind the choice of numbers is rather complex. Those interested should see [Knuth]. We suggest the use of the following values :

COEFF0 = '1254635 (loaded LEFT_ADJUSTED)
TERM_0 = '660623

TERM_1 controls where we start in the sequence. The starting value of TERM_1 may be anything at all. If you wish to repeat the sequence of random numbers simply start TERM_1 off with the same value as the starting value of the sequence you wish to repeat.

For the record we give the following justification for the above choice of numbers, which may be safely ignored by the average user. The additive term is the nearest prime to the value recommended in [Knuth] Section 3.3.3. The multiplier was subjected to the spectral test, written by Bill Gosper on MACSYMA (see [Gosper]). The resulting values of C2 through C9 are:

C2= 2.746 C3= 4.310 C4= 1.322 C5= 2.631
C6= 3.135 C7= 0.3052 C8= 1.284 C9= 3.456

Other multipliers may have better values but these are quite good.

There is a sort of degenerate use of uniform noise mode that can be used to create ramps. For instance, if TERM_1 is set to 1, COEFF0 also to 1, and TERM_0 is cleared, when the modifier is started it will count up by one through 20 bits, wrap around and start over. Change TERM_1 for any steeper slope, and set TERM_0 to get it to start at any phase within the ramp. Notice that the ramp will be appear left adjusted in sum memory. This is ok for passing indicies to a modifier in table lookup mode, since it has provisions for right justifying the index before addressing memory.

17.13. Triggered uniform noise

TR_U_NOISE

Result ← COEFF0*TERM_1 + TERM_0. If B_IN*COEFF1 ≠ 0 then
TERM_1←result.

A new noise value is calculated as in U_NOISE above only when the "trigger", B_IN*COEFF1, is not 0. This allows the production of noise values to be dependent on some state in the box rather than requiring a command to get it going or stopped. Priming of the terms of this procedure should be done according to the rules for U_NOISE above.

17.14. Invoke delay unit

INVOKE_DELAY_UNIT

```
(B_IN(low order 5 bits) ← <delay unit number to send samples to>;  
result ← TERM_0 + TERM_1*COEFF0;  
TERM_0 ← <delay memory value read from delay unit>;  
<value sent to delay unit> ← A_IN + <delay memory value>*COEFF1;
```

This is a little tricky, but it goes like this: To couple delay memory to sum memory requires two things: a way to get things into and out of sum memory, and a way to get things into and out of delay memory. There must also be a way to get these two interfaces to know about each other. A modifier, when put in this funny mode handles the bulk of this interface: it reads sum memory, does the funny algorithm on it shown above which I'll get to in a moment, and then passes this processed sample to a particular delay line that it knows about (stored in B_IN for this mode). The delay unit does the clerical tasks of addressing the delay memory, plunking down samples received and shipping others back to the controlling modifier. The delay units are discussed in more detail on page page 61.

The modifier in this mode performs arithmetic to implement a comb or allpass reverberation section. The action of the modifier consists of a feed-forward term and a feed-back term where COEFF0 controls the gain on the feed-foreward term, and COEFF1 controls the gain on the feed-back term. If the feed-foreward term is between 0 and -1, and the feed-back term is between 0 and +1, then the algorithm represents an allpass resonator. If the feed-foreward term is 0 and the feed-back term is non-zero, then it is a comb resonator. In addition, if they are both 0, then the unit acts as a simple one-shot delay line.

Section 18

Modifier filtering algorithms

Various flavors of filtering are available. Here is the actual algorithms, starting with a SAILish version of the code performed. Section 18 has a little more on how to use them, written by Andy Moorer.

18.1. One pole

Called as **ONE_POLE** mode to a modifier.

```
Result←COEFF1*TERM_1 + COEFF0*B_IN;
TERM_1←Result;
```

Or, the formula could be given as:

$$\text{Result}[k] \leftarrow \text{COEFF1} * \text{Result}[k-1] + \text{COEFF0} * \text{B_IN}[k],$$

where k is the number of the current sample, $\text{B_IN}[k]$ is the location of the current input sample, COEFF0 and COEFF1 are gain factors. The result is formed of $\text{TERM_1} * \text{COEFF1} + \text{B_IN} * \text{COEFF0}$, then TERM_1 is replaced by the result, to be processed on the next pass. In this way TERM_1 becomes the $k-1$ 'th sample.

18.2. One zero

ONE_ZERO

```
Result←TERM_1*COEFF1 + A_IN*COEFF0;
TERM_1←A_IN.
```

Or:

$$\text{Result}[k] \leftarrow \text{A_IN}[k] * \text{COEFF0} + \text{COEFF1} * \text{A_IN}[k-1].$$

18.3. Two poles

TWO_POLES

```
Result←COEFF1*TERM_0 + COEFF0*TERM_1 + A_IN;  
TERM_0←TERM_1;  
TERM_1←Result;
```

This is like saying

```
Result[k]←COEFF0*Result[k-1] + COEFF1*Result[k-2] + A_IN.
```

18.4. Two zeros

TWO_ZEROS

```
Result←TERM_0*COEFF1 + TERM_1*COEFF0 + A_IN;  
TERM_0←TERM_1;  
TERM_1←A_IN;
```

This is like saying

```
Result[k]←A_IN[k] + COEFF0*A_IN[k-1] + COEFF1*A_IN[k-2].
```

18.5. Two poles COEFF0 variable

TWO_0POLES

```
Result←TERM_0*COEFF1 + TERM_1*COEFF0 + A_IN;  
TERM_0←TERM_1;  
TERM_1←Result;  
COEFF0←COEFF0 + B_IN;  
<increment k and go to LOOP>.
```

18.6. Two poles COEFF1 variable

TWO_1POLES

```

Result←TERM_0*COEFF1 + TERM_1*COEFF0 + A_IN;
TERM_0←TERM_1;
TERM_1←Result;
COEFF1←COEFF1 + B_IN;

```

18.7. Two zeros COEFF0 variable

TWO_0ZEROS

```

Result←TERM_0*COEFF1 + TERM_1*COEFF0 + A_IN;
TERM_0←TERM_1;
TERM_1←A_IN;
COEFF0←COEFF0+B_IN;

```

18.8. Two zeros COEFF1 variable

TWO_1ZEROS

```

Result←TERM_0*COEFF1 + TERM_1*COEFF0 + A_IN;
TERM_0←TERM_1;
TERM_1←A_IN;
COEFF1←COEFF1+B_IN;

```

18.9. A little digital filtering theory

(...never hurt anybody). Now I bet you want to know how to use these things, right? Well, filters are entirely specified by their frequency responses, so let's take a look: In what follows, w is the radian frequency ($2\pi f$) of a pure sinusoid applied to the filter to determine its frequency response. T is the sampling interval, or $1/\text{clock_rate}$.

$$\text{ONE POLE: } H(w) = \text{COEFF0}/\sqrt{1+\text{COEFF1}^2-2*\text{COEFF1}*\cos(wT)} \quad \text{Equ. 13}$$

$$\text{ONE ZERO: } H(w) = \sqrt{\text{COEFF0}^2+\text{COEFF1}^2+2*\text{COEFF0}*\text{COEFF1}*\cos(wT)} \quad \text{Equ. 14}$$

$$\text{or } H(w) = \text{COEFF0}*\sqrt{1+R^2+2*R*\cos(wT)} \text{ where } R=\text{COEFF1}/\text{COEFF0} \quad \text{Equ. 15}$$

$$\text{TWO POLES: } H(w) = 1/\sqrt{(1-\text{COEFF0}*\cos(wT)-\text{COEFF1}*\cos(2wT))^2 + (\text{COEFF0}*\sin(wT)+\text{COEFF1}*\sin(2wT))^2} \quad \text{Equ. 16}$$

$$\text{TWO ZEROS: } H(w) = \sqrt{(1+\text{COEFF0}*\cos(wT)+\text{COEFF1}*\cos(2wT))^2 + (\text{COEFF0}*\sin(wT)+\text{COEFF1}*\sin(2wT))^2} \quad \text{Equ. 17}$$

These expressions won't mean much without some explanation. Let's look at the ONE POLE case. Since $\cos(0)=1$, $H(0)=\text{COEFF0}/|1-\text{COEFF1}|$. At half the sampling rate, $wT=\pi$ so $\cos(wT)=-1$,

thus $H(2\pi\text{clock_rate}/2) = \text{COEFF0}/(1+\text{COEFF1})$. So we see that the frequency response starts at $H(0)$ and varies roughly sinusoidally to $H(2\pi\text{clock_rate}/2)$. If $\text{COEFF1} < 1$, then this means that for positive COEFF1 , this is a low-pass filter (accentuates frequencies around zero) and for negative COEFF1 , it is a high-pass filter (depresses frequencies around zero). Note that if $\text{COEFF1} = 1$ or -1 , you get a filter the frequency response of which goes to infinity either at 0 or $2\pi\text{clock_rate}/2$. This is not recommended.

For ONE ZERO, it is the same deal except that it is the ratio of COEFF1 to COEFF0 that sets the frequency response and the sign is inverted. Positive COEFF1 makes a high-pass and negative COEFF1 makes a low-pass.

TWO POLES: Now the really interesting (and hard to explain) stuff comes when we get to two of everything. (double your pleasure, double your fun!...) The only way to deal with a two pole or two zero filter is to put it in canonical form (that word again!). Here is the canonical form for the TWO POLES case:

$$H(w) = G/\sqrt{(1-2*R*C*\cos(wT)+R^2*\cos(2wT))^2+(-2*R*C*\sin(wT)+R^2*\sin(2wT))^2} \quad \text{Equ. 18}$$

where $G=1$, $\text{COEFF0}=2*R*C$, and $\text{COEFF1}=-R^2$.

Without further explanation, this is a resonator (formant filter). Its resonant frequency is determined by C , which is $\cos(2\pi fT)$. Its bandwidth (sharpness) is determined by R which must be less than one for stability. Typical values would be $R=.95$ for a fairly sharp filter and $R=.999$ for a filter that is so sharp that it will "sing" at its resonant frequency almost by itself. To use such a filter, you choose the resonant frequency, f , choose the bandwidth, R , the compute $\text{COEFF0}=2*R*\cos(2\pi fT)$ and $\text{COEFF1}=-R^2$. This will give you that filter.

We have, in this description, sidestepped the issue of scaling. The gain of this filter at resonance is

$$H(2\pi f) = G/\sqrt{4(1-\cos(2\pi fT))((1-R)^2-(1-R)^3)+(1-R)^4}. \quad \text{Equ. 19}$$

This looks a lot better if you substitute something, like D , for the term $(1-R)$:

$$H(2\pi f) = G/\sqrt{4(1-\cos(2\pi fT))(D^2-D^3)+D^4}. \quad \text{Equ. 20}$$

For most purposes, you can normalize a filter by multiplying it by $(1-R)$. For higher frequencies, you need a smaller factor. The only problem is that there aren't any more multiplies left in a modifier (no COEFF2 , just COEFF0 and COEFF1). This means that you have to scale the signal going into the filter by $(1-R)$ first. Don't forget this, because otherwise you will get lots of overflow.

Note also that COEFF0 and COEFF1 are really less than 1.0. This being the case, he said inquisitively, how can we possibly synthesize $2*R*C$, which can be as large as 2, using fractional arithmetic? Well, remember the SCALE terms? That's how! The right four bits of the mode are the scaling for the multiplies. You can scale the result of the multiplication by COEFF0 and the

multiplication by COEFF1 by a power of two. If we call the right 4 bits of the mode AABBB, then AA or BB set to 0 specifies no scaling, 1 means scale by factor of 2, 2 by factor of 4, and 3 by 8. AA is the scaling for the multiply with COEFF1, BB is the scaling for the multiply with COEFF0. This means that we can get 2RC by setting COEFF0 to RC and setting the scaling for the multiply by COEFF0 to 1 (factor of 2).

Now normalizing the response at resonance to 1 may or may not be exactly the right thing to do. The problem is that this only makes sense if your signal is a narrow band signal, like a pure sinusoid or something. Otherwise, normalizing by the gain at resonance is too much attenuation. Unnormalized gives too much gain and normalizing by the gain at resonance is too much attenuation, so anywhere in between (like multiplying by $(1-R)$) is probably reasonable. Figures 10 and 11 show filter responses for two-pole resonators at three different center frequencies and a number of different bandwidths. Figure 10 shows a normalized filter. Notice that the gain is now 0 dB (1.0) at the center frequency, but for high values of R, the peak is really narrow. Figure 11 shows the unnormalized filter responses. Note that at high values of R, the filter can emphasize the signal by as much as 40 dB.

Normalizing things with zeros rather than poles is much easier. The maximum response for a filter with zeros occurs either at zero frequency or at half the sampling rate, and this maximum is guaranteed to be less than 4.0 (for a real notch filter, rather than two first-order filters multiplied together). Thusly, you can pretty much ignore the scaling of an all-zero filter because you probably won't get too screwed by it. Figure 12 shows the responses of two-zero filters (or antiresonators, as we might call them).

And not to leave out the first order filters, figures 13 and 14 show the responses of one pole and one zero filters. Here the maximum and minimum values occur only at zero and at half the sampling rate. In all these plots, the horizontal axis represents frequency from zero to half the sampling rate. You can see that positive values of R for the one pole case represent low-pass filters. The frequency responses of all-pole filters are just the inverses of those for all-zero filters. Isn't that handy?

FPIX.XGP[DOC,MUS] goes here. (FIGURE 9)

(FIGURE 10)

(FIGURE 11)

DATE!

DRAFT

(FIGURE 12, FIGURE 13)

Section 19

Delay Units - Delay Memory

Delay memory is a read/alter memory with up to 65k (65,536 decimal) of 20 bit words. This is a common pool of memory, so that any part of it is addressable by each of the delay units.

The 32 delay units are the link between delay memory and modifiers.

(Delay units and delay memory are called the "delay memory option" in the [Specification]. The option is either no delay memory and therefore no delay units, or increments of 1k of delay memory up to the maximum of 65k and all 32 units. Stanford has 48k words of delay memory.)

The signal path using delay memory is like this: a modifier reads sum memory, passes the sample to a delay unit which either writes the sample somewhere in delay memory, or uses it as an address to into delay memory, depending on the mode of the delay unit. In either case, the value from the referenced memory cell is sent back to the modifier via its associated delay unit.

Modifiers are hooked up to delay units by specifying them to be in INVOKE_DELAY_UNIT mode. The address of the delay unit the modifier will communicate with is put in the B_IN term of the modifier. Doing so sets up the communication link between that modifier and the named delay unit (the delay unit's connection back to that modifier is implied from this in the hardware).

The delay units thus invoked have two possible modes, TABLE_LOOKUP and DELAY_LINE, discussed below. The following lists the delay unit fields and modes. The function of the fields is altered by the mode of the delay unit, so their full description is given with their modes, below.

Id	Definition	Description

P	MODE	Operating mode.
Z	DELAY_LENGTH	# of words this unit addresses.
	SCALE	table address scale factor.
Y	INDEX	Running index into delay memory.
X	BASE_ADDRESS	Base address in delay memory.
Modes:		
	D_INACTIVE	dead.
	DELAYLINE	see below
	TABLE_LOOKUP	see below
	ROUND_TABLE_LOOKUP	see below

Fig. 15

19.1. Delay line mode

DELAYLINE

```
result←DM[INDEX + BASE_ADDRESS];  
DM[INDEX + BASE_ADDRESS]←<data from modifier>;  
INDEX←INDEX + 1;  
if INDEX>DELAY_LENGTH then INDEX←0;
```

This is used to read/write successive locations in delay memory for reverberation. The delay unit references memory starting from BASE_ADDRESS for DELAY_LENGTH words. The INDEX is incremented by the hardware every pass, modulo DELAY_LENGTH (that is, the INDEX is set to 0 when its value exceeds DELAY_LENGTH). The delay unit reads the location pointed to by BASE_ADDRESS+INDEX, and sends it back to its modifier, then takes the new sample from the modifier, and puts it in the now vacant slot, then increments INDEX.

19.2. Table lookup mode

TABLE_LOOKUP

```
INDEX←<value from modifier> right-shifted (bringing in zeros)  
by SCALE;  
IF INDEX>LEN then set CONI error;  
result←DM[INDEX + BASE_ADDRESS];
```

In this mode, the modifier sends the delay unit an index into its chunk of delay memory, and the unit sends back what it finds. This is a read only operation, the value in memory can not be changed. The index sent from the modifier can be scaled right, bringing in zeros, up to 16 places before being used to address the memory. (Indices for table lookup are most often generated in fractional form, i.e., left adjusted in sum memory, hence the provision for right shifting to reduce it to the right range.)

19.3. Table lookup - rounded

ROUND_TABLE_LOOKUP

This is the same as for TABLE_LOOKUP above, but the modifier index received is rounded after shifting, then used to address delay memory. This gives you 3dB better resolution than straight table lookup. Resolution is to 1/2 the LSB, rather than just the LSB in plain

TABLE_LOOKUP.¹

One thing to watch out for is if the address is rounded up off the last word of the table. The two options in designing the machine were to wrap around or to allow the one word overflow. The latter was implemented for the reason that, if you were doing a square root table, for instance, wrapping around wouldn't give you any kind of a right answer. This means that you'd better claim one extra location at the top of your table and stick in some appropriate value.

19.4. Other phenomena

There is a built in 3 pass delay in getting samples through the delay unit's pipe. This means, for instance, in using DELAYLINE mode, the length of delay memory should be 3 less than the desired delay because you get them for free in the delay line. In table lookup modes, the effect is that you don't get the sample back until three passes after you specify a new address.

Please note, the hardware does no automatic clearing of delay memory. It therefore will return junk on the first time through. Also, it requires an additional three passes to return data from the delay unit, so the real turn around time is DELAY_LENGTH + 3.

For an example of calling a delay unit in DELAYLINE mode see page 109.

¹* The next more accurate way to go would be to interpolate between the two adjacent values in the table.

Section 20

Instruction and command structure

There are two types of instruction to the Synthesizer: those which configure the Synthesizer as a whole and those which set values in the individual processing elements. There are some which don't seem to fit either category, but basically they can be broken down on the basis of whether they should be in the hands of the general user or should be under system control.

The term "command" will be used throughout to refer to user-settable parameters, while the term "instruction" will be used for parameters to be set under system control.

Section 21

Processing element commands

Commands generally consist of three parts: the processing element which is to be referenced, which field of the element is to be set, and the value.

Each command to the synthesizer takes the form of a single 32 bit word. If the word length of the CPU is greater than this (the Stanford CPU is a 36 bit machine) the commands are packed right adjusted in the word and the high order bits are thrown away by the Synthesizer.

Commands generally have three fields: a 20 bit data field on the left side, a 4 bit operation code (op-code) in the middle, and a 7 or 8 bit processing element field on the right side. The processing element field specifies the index number of the element in question, but doesn't specify which kind of element. Occasionally, there are other bits in various places in a particular command that do small tasks.

The op-code tells which parameter the data is for. These parameters are discussed at length in the preceding sections of this manual. There is one unique code for each parameter type. Since any one parameter type only shows up in one kind of processing element, specifying which parameter to set tells us which type of processing element it is for as well. The index can then refer to the individual instance of that element.

Sometimes, where the field of a particular parameter is short, two or more commands can be packed in one word. This is the case in generators where ASYMPTOTE and SUM_MEMORY can be specified in one command. In these instances, there are special bits available to disable loading one or the other.

There are other instances in modifiers where there are two parameters which are identical in function (such as the two coefficients, the two running terms, etc.). In this case a basic op-code describes both, and a bit is provided to determine which one is to be loaded into the processing element.

The data field part of these commands is restricted to 20 bits, however there are some processing element parameters that are wider than this (namely, FREQUENCY, EXPONENT, COEFF0 and COEFF1). So, there is a special register provided to hold the excess for those fields that need it. This is called the DX (for data extension) register. There is a special command (described later) to load this register, and in each command type which needs it there is a bit which says whether or not to append DX to the 20 bit data field. If loading DX is indicated, then the data in DX is appended to the right end of the 20 bit data field in the command. This means that DX is meant to load the low order bits of the long field, and it also means that the data to be put in DX should be put in left adjusted so it will butt up against the bottom of the 20 bit datum in the command. Since DX loads the low order bits of these fields, if the full precision of the field is not needed, DX need not be used. In some cases this has implications for how the value of the field is calculated (see section 15).

When DX is used, it is always put to zero (cleared) automatically after the value is read from it. There are a couple of other commands which have bits that allow clearing of other related parameters, namely, modifier MODE commands can clear TERM_0 and A_IN/B_IN commands can clear TERM_1. TERM_0 and TERM_1 can, of course be set explicitly by their own commands.

21.1. Miscellaneous command

This is a clearing house for odd ducks. This command can be used to set one of:

DX: Load DX from the 20 bit data field. The value loaded should be the low order bits of the parameter its is being used for loaded left adjusted in the DX field. For instance, if frequency of a generator is to be specified as a full 28 bit field, put the low order 8 bits left adusted in the 20 bit DX data field, and the remaining high order bits should go into the FREQUENCY data field, right adjusted in its 20 bits. The FREQUENCY command should come immediately after loading DX in the command stream, since any intervening command that could reference DX would get the data meant for this FREQUENCY field instead and all hell would break loose.

TTL_A or TTL_B: Load these TTL buffer registers from the high order 16 bits of data. Don't ask me why these are here. TTL_A has no defined function whatsoever (could be used to control Christmas lights or something on a real-time control panel, or run a light-show, or...). TTL_B, however has a further oddity in that it is used to set the analog output filters (see below for more on these).

Clear all wait bits, Clear all pause bits: See section 9 for an explication of these features.

Stop clock: Not an old TV game show, but a way for the user to stop the box. There is a CONO instruction to stop the Synthesizer, but this is a way for the command stream to be able to shut it down when the user wants. There is no comparable command to turn it back on (a CONO instruction must be issued to do this). This is actually a very important command to have. There is no way for the CPU to know when processing is done. Even in the simple case where the box is just snarfing down a command stream there is ambiguity. The CPU knows only when it has exhausted the last buffer of such commands, but this is no indication that the commands themselves have been executed just by being read into the Synthesizer. (For instance, they could all be long LINGERS that could cause the box to run on for hours.) The only way to handle it is this: the user, by convention, puts a STOP command at the end of every command list; when the CPU has received the last "command buffer done" message from the Synthesizer, it begins periodically polling it using CONI. When it detects the clock running flag off, it knows processing has stopped.

Analog Filters: Loaded from right 4 bits of data. There are three modes for the filters, UNFILTERED, filtered, and no change. If the filters are selected, there are 4 possible choices of cutoff frequency: f0, f1, f2 and f3. In the Stanford machine, these translate into lowpass filters with a rolloff of 18 dB per octave, with their respective 3dB points at 4.5 kHz, 9, 13.5 and 18 kHz. UNFILTERED mode is basically straight through (flat to 200 kHz).

21.2. Delay command

This command is used to set all parameters of delay units. It has the basic division of data, op-code, special bits and element index. The special bits augment the op-code to indicate which field to set, `BASE_ADDRESS`, `INDEX`, `SIZE`, `SCALE`, or `MODE`. The 20 bit data field is subdivided into two fields, a high order 16 bit part, and a low order 4 bit part. The 16 bit part is taken as data for loading the `BASE_ADDRESS`, `INDEX` and `SIZE`. In the `SIZE` command, the low 4 bit subfield is used to set the `MODE`. When one sets the `SIZE`, one must also set the `MODE`, there are no disable bits to load just one, and for a reason: if the `MODE` bits specify `DELAY_LINE`, then the 16 bit `SIZE` data is taken as the size of the delay line table. If the `MODE` is `TABLE_LOOKUP`, then the size of the table is moot (since it will be determined by the range of index presented by some other processing element at runtime), so instead the low order 4 bits of the 16 bit subfield are taken as the `SCALE` factor to use to shift the index received at runtime to the right before actually addressing the memory.

Notes: Setting the `BASE_ADDRESS` also clears the `INDEX`. The `INDEX` must be the one's complement of the desired number. `SIZE` must be 1 less than the length desired. For more details on the operation of delay units, see section 19.

21.3. Timer command

This command has two functions, to cause the Synthesizer to stop processing commands for a specified period, and to clear or set the pass counter. It has a 20 bit data field.

`LINGER`: stop processing commands until the pass counter equals the data. Also called "absolute linger".

`PASS_CLEAR`: clear (set to zero) the pass counter, then `LINGER` until pass counter equals data. Also called "relative linger".

`CLEAR_PASSES_LINGER`: set the pass counter from data.

Note: There is a restriction on the range of the data that can be given in a `LINGER` command. There is a trap in the hardware for detecting command overrun, such that if a `LINGER` command specifies waiting for a pass that is in the recent past, it assumes that the passing of commands has outstripped the ability of the box to soak up commands fast enough. The range of numbers tested for this is between 1 and 4096 before the current pass. This means that a `LINGER` cannot be issued for greater than $2^{20}-2^{12}-1$ passes or it will be interpreted as one of these conditions. The Synthesizer can interrupt the CPU when such cases are detected, if desired.

Note also, that while lingering, the box continues to take up time for the update commands even though it doesn't process them, that is, the total number of ticks per pass stays the same.

For more details, see page 14 and 79.

21.4. Ticks command

This sets the number of processing and total ticks from the 10 bits of data.

PROCESSING_TICKS: (to quote from the Specification) "designate the highest-numbered processing tick per pass (should not exceed 255)".

TOTAL_TICKS: "designate next-to-highest-numbered tick (processing plus overhead plus update) per pass". Note: we are not setting a range, but a specific, absolute element number or tick number. Thus, if you want all 256 generators to run, set **PROCESSING_TICKS** to 255, because the first processing element is number 0. If you want a total of 384 ticks per pass, set **TOTAL_TICKS** to 382. This is because the first tick begins at 0, and we also are told to specify the "next-to-highest" tick. This is a net reduction of two! Think about it...

Note: the issuing of these commands must be done in a particular order when cranking up the Synthesizer from an unknown state. See section 28.

Section 22

Synthesizer instructions

In section 20 I discussed the division of control of the Synthesizer into "instructions" and "commands". A full treatment of instructions is beyond the scope of this manual, but a summary of their functions will be given. In this section I will assume some knowledge of priority interrupt processing.

The control of the overall status of the Synthesizer is determined by the instructions issued to it, called CONOs, (for CONditiOnsOut, out being the direction always with respect to the CPU). When the Synthesizer has something to report back to the CPU, it typically interrupts the CPU, which then issues an appropriate CONI (for CONditiOnsIn). The Synthesizer then responds with the appropriate information to tell the CPU what it wanted to say.

When the CPU wants to give the Synthesizer some data, it uses the DATAO class instructions. If the Synthesizer has data for the CPU, it uses the DATAI instructions.

This message protocol takes place over the I/O bus, which is a relatively slow (compared to the DMA) path, and is used only for infrequent or low bandwidth communication.

Ordinarily, a device has one set of CONO/CONI/DATAO/DATAI instructions, but this beast has four! These correspond to the A, B, C or D following the instructions in the summary below.

Also, there is usually one interrupt channel for most devices, but the box has two. The first, PIA-A is used to report general conditions, PIA-B is specifically to inform the CPU that it has run out of commands. The unbroken flow of commands is taken as being important enough that it should have a channel all to itself.

Here then is a summary of the instructions:

CONO-A 18 bits sets overall status, such as starting/stopping clock, setting PIA channels, master reset and diagnostic readback address.

CONO-B 18 bits sets miscellaneous status, such as resetting memory error flags and the like, and enabling/disabling stop or interrupt on various causes, including arithmetic errors and DMA word counts exhausted.

DATAO-A 32 bits sends command to be performed. Individual commands can be sent over the I/O bus, instead of having to stuff them through the DMA. Very convenient for debugging.

DATAO-B 36 bits sets core address (CA) or word count (WC) for commands, read and write data. (CA for these should be written before word count since a WC becomes not exhausted as soon as it is written into.)

DATAO-C 20 bits for diagnostic purposes, only when running, sets write-data buffer from bits 4-23.

DATAO-D 36 bits writes data into Delay Memory at a specified address, used for preloading tables. Data overwritten can be retrieved with DATAI-D. While this can be used anytime, it is best restricted to preloading tables as it is not very fast, it is not synchronous with the tick level processing, and it steals from the ability of the delay units to read Delay Memory.

CONI-A 20 bits reads overall conditions, such as which channel interrupted, the sort of arithmetic or memory error encountered, or WC-MA exhausted.

CONI-B 16 bits reads cause of interrupt, including, for arithmetic errors, which processing element barfed, how, and the tick number when it occurred.

DATAI-A 20 bits diagnostic readback (only when not running). The diagnostic readback can reveal the gizzards of the box to the knowing eye. If there is a hardware bug, this is how you trace it down (definitely for wizards only!).

DATAI-D 20 bits reads Delay Memory data overwritten by the most recent DATAO-D.

CONI-D since it takes time for the box to write the data obtained with DATAO-D into Delay Memory, fetch the value there and put it where the next DATAI-D will find it, the CPU must know how long to wait after doing the DATAO-D before doing the next DATAI-D. CONI-D tells all.

Section 23

The Stanford software

23.1. Overview

The software developed at Stanford for the Synthesizer is a three level system.

The lowest level is the command assembler. It consists of a set of routines which configure commands and assemble them in one or more output streams. It also has tables and routines for doing allocation/deallocation of processing elements. It is written in FAIL, the local version of PDP-10 assembly code. Only the low level code will be fully discussed here.¹ However, for an appreciation of the overall structure, a brief description is given below.

The low level also includes two debugging programs: DAB and UNDAB which stand for DissAssemble the Box and UNDisAssemble the Box, respectively, and what they do is described in detail later. One final low level program is called MERGE which allows for pre-compiled command streams to be combined.

The upper two levels are written in SAIL, which is an ALGOL based language developed at Stanford, and which contains a very powerful parallel process runtime mechanism. The intermediate level consists of a library of useful routines mostly having to do with providing combinations of calls on the low level routine to configure the Synthesizer to do standard primitive tasks. There are also routines here which support the high level, having to do with the SAIL process runtimes.

The high level code consists of groupings of intermediate level routines to make complex configurations called "instruments". These are invoked from a scanner/parser which reads a "score" specifying the calling sequence and values which the instruments are to use to do their thing. There are facilities for controlling the running of instruments from real-time input devices as well.

23.2. Low level routines

The low level code (henceforth referred to as LOWER) consists of two parts.

There is the assembler and processing element allocation object code in one file and it contains the routines described immediately below. They handle the set up of data paths in the box and the passing of parameters. This is also where the processing element allocation scheme lives.

¹ Documentation for the upper two levels exists, but is unpublished. Contact CCRMA for further information.

Another file contains the macro definitions of all the terms such as `ANGLE` and `TERM_1` that have been discussed in other parts of this document. This second file has two uses: 1) to define the macros which can be used with the low level routines, and 2) to tell the compiler of the user's program (`SAIL`) how to link up to the procedures in the external low level routine package.

At Stanford, these files are named `LOWER.FAI[SAM,MUS]` (object code for the assembler), `LOWER.REL[SAM,MUS]` (relocatable binary file), and `LOWER.DEF[SAM,MUS]` (contains macros and External declarations). (An additional file `LOWDEF.HDR[SAM,MUS]` contains only those macros liable to be useful to the user and can be substituted for `LOWER.DEF`.)

Section 24

Low level allocation routines

24.1. GET

External Simple Integer Procedure GET(integer id).

(In this and all that follows, SAIL constructs will have their first letter capitalized, constructs from LOWER.DEF will be entirely capitalized.)

This procedure is used to claim a processing element of type id. There are macro names to use for id are:

Definition

 ID_GENERATOR
 ID_MODIFIER
 ID_DELAY
 ID_SUM_MEMORY

Fig. 16

GET has two actions: it finds a free element of the type specified in id by looking on arrays reserved for the purpose which keep track of which elements are in use. If it finds one it marks it as in use and returns the address of the element.

GET keeps track of which elements have been claimed by referencing the processing

There is one more processing element type: ID_INVALID which is returned by GET if there are no more elements of the type requested.

If GET can't make sense out of id, it calls the error handler.

For an example, if tweet is an integer, saying: (in SAIL) `tweet←GET(id_generator)` will return a pointer to the first free generator in tweet. That generator is marked as being used, and no future call to GET will return a pointer to this element until it is given back to the unused pool. Likewise: `tweet←GET(id_modifier)` returns a pointer to a modifier, and `tweet←GET(id_delay)` does the same for a delay unit.

GET will always return the lowest numbered free element of the type specified in id (if it can).

This system works fine for claiming processing elements, but falls short of adequate for two types of resources: sum memory and delay memory.

The problem of sum memory comes about because one typically wants to select one particular coordinate or other. However, GET(ID_SUM_MEMORY) only returns the absolute address of the lowest numbered free sum memory location regardless of coordinate. To GET a specific coordinate, one must add one of the following qualifiers from the next table to the SUM_MEMORY mnemonic (using simple addition). For example GET(ID_SUM_MEMORY+THIS_MOD_PASS) returns the first available this pass modifier side sum memory location. By far the two most used quadrants of sum memory are LAST_MOD_PASS and LAST_GEN_PASS, so two special mnemonics exist for them: MOD_SUM and GEN_SUM. So, for example, GET(MOD_SUM) will return the first free modifier last pass sum memory location.¹ element tables (see below).

Definition	Function
THIS_MOD_PASS	gets modifier this pass sum memory.
LAST_MOD_PASS	gets modifier last pass sum memory.
LAST_GEN_PASS	gets generator last pass sum memory.
THIS_GEN_PASS	gets generator this pass sum memory.

Fig. 17

The allocation of delay memory is handled by a separate intermediate level routine, DM. See [Schottstaedt].

24.2. GIVE

External Simple Procedure GIVE(Integer element).

This routine puts the named element back on the free list. The actual value of the argument is unchanged, it still points to the element. However, the allocation routines consider that the element is no longer in use and might very well give it to someone else which could result in some quite perplexing results. Be sure you don't try to use a pointer after a GIVE. Note also that one is never guaranteed to GET the same element back after it is released. A sample command: GIVE(tweet).

¹ This scheme is sufficiently clumsy that a separate allocation routine for sum memory has been implemented in the intermediate level code. However, that routine is only a front end for GET. See [Schottstaedt].

24.3. Processing element tables

GET and GIVE keep track of the type and use status of the elements they manage using these arrays.

These arrays are made available to the user so that one can write one's own procedures to do the book keeping for the processing elements instead of using GET and friends.

Notice on the following tables that there is one big array from 0:671 and several smaller ones that add up to a total range of 0:671 also. Actually, they overlap, such that the location referenced by `unit_generators[0]` is the same as that addressed by `generators[0]`, and location `unit_generators[256]` is the same as `modifiers[0]`, etc. These small arrays fit inside the `unit_generators` array in the order given below. Also notice that `all_sum_memory[0]` is the same as `gen_sum_memory[0]`, and `all_sum_memory[128]` is `mod_sum_memory[0]`.

An element's number is taken as its address on the `unit_generators` array, so that, for instance, the first modifier is referenced as processing element number 256.

The arrays are allocated as follows:

External integer array `unit_generators[0:671]`.

All the generators, modifiers
delay units and sum memory addresses are listed here
as to availability.

External integer array `generators[0:255]`. Just the generators.

External integer array `modifiers[0:127]`. Just the modifiers.

External integer array `gen_sum_memory[0:127]`.
Just the generator sum memory.

External integer array `mod_sum_memory[0:127]`.
Just the modifier sum memory.

External integer array `gen_this_pass_sum_memory[0:63]`;
external integer array `gen_last_pass_sum_memory[0:63]`;
external integer array `mod_this_pass_sum_memory[0:63]`;
external integer array `mod_last_pass_sum_memory[0:63]`;
Individual quadrants.

External integer array `all_sum_memory[0:255]`. Both sum memories.

External integer array `delays[0:31]`. Just delay units.

Fig. 18

Currently, the tables are initialized to 0, and GET marks a claimed processing element location with -1. GIVE returns the value to 0.

24.4. DECODE

External Simple Integer Procedure DECODE(Integer unit). DECODE returns the type of processing element, as follows:

macro name	integer value
ID_GENERATOR	0
ID_MODIFIER	1
ID_DELAY	2
ID_SUM_MEMORY	3
ID_INVALID	-1

24.5. RELATIVE

External Simple Integer Procedure RELATIVE(Integer unit).

When passed an id that has been GETed, RELATIVE returns the index of that unit relative to the base address of that type unit, i.e., measured from the base of its own processing element array, not the global unit_generators array.

It returns ID_INVALID if it can't make sense out of the request.

Section 25

Command assembly

There are four routines which activate the assembler. The only difference between them is in how many parts there are to specify about the command to be assembled. There are things in the box that require specification of three parts, four parts, two and one parts, for which one uses BIND, BIND_FIELD, SET_FIELD and SET_MODE respectively. BIND and BIND_FIELD are used exclusively for setting parameters to processing elements, SET_FIELD and SET_MODE handle all the rest.

25.1. Bind

External Simple Integer Procedure BIND(Integer element, field, val).

This is the heart of the assembler. It has three functions: a) to assemble parametric data into fields of commands for processing elements, b) to assemble commands to set the input and output ports for specific sum memory locations, and 3) to assemble commands that set the running status of processing elements. The first argument to BIND specifies which processing element we are addressing, the second determines the field in question, and the last is the value we will deposit in that field of the command.

For instance:

```
BIND(tweet, FREQUENCY, 440*228/clock_rate)
```

sets the generator referenced by tweet to have a frequency of 440 Hz (That is, it puts $440*2^{28}/\text{clock_rate}$ in FREQUENCY).

BIND is also used to assemble commands for the output and input addresses of processing elements. For instance, the following sequence will link two generators:

```
Integer sum_loc;
sum_loc←GET(GEN_SUM);
BIND(tweedle,SUM_MEMORY,sum_loc)
BIND(didi,FM,sum_loc)
GIVE(sum_loc);
```

The first call to BIND specifies that we wish to issue a command to the box to set tweedle's output will go to the location addressed in gen_sum. The second call to BIND will assemble a command which causes didi to read location gen_sum for its frequency modulation input. This is an instructive example for another reason: notice that we used the mnemonic GEN_SUM for the

call to GET which expands to ID_SUM_MEMORY + LAST_GEN_PASS, and yet we know that the input to a generator comes from THIS_GEN_PASS. Well, BIND knows that too, and figures out what we intend. The general convention is to specify sum memory always in terms of last pass, and let BIND figure it all out.

An example of setting the running status of a processing element might be: BIND(fut,MODE,A_RUNNING). Valid terms for the element argument to BIND are: all generators, modifiers or delay units. Valid terms for the field argument are: all parameters of the above elements, such as FREQUENCY, RATE, EXPONENT, MODE, A_IN, COEFF1, etc. The field must obviously refer to a field that the element has, i.e., don't try to set COEFF0 of a generator. Valid values for the val argument depend on the field being set. The values must fit the hardware bit width for that field (e.g. 28 bits maximum for FREQUENCY, etc.). Special trapping is done to detect magnitude out of bounds and an the error handlers are invoked if any field is out of bounds.

A further word is needed on the subject of magnitude overflow. This can be skipped, but you may regret it some day. All the word lengths in the synthesizer are smaller than the 36 bit representation of the val argument to BIND (typically they are 20 bits, sometimes 12, 24, 28 or 30 depending on the parameter being set). This raises the obvious problem that we are able to represent larger magnitudes in the val argument to BIND than can be fit into a particular hardware register. There is a subtler problem that arises with trying to represent negative data. For instance, for a 20 bit hardware parameter register, '2000000 represents -1. But remembering that the val argument used to produce this is a signed 36 bit two's complement integer, that would make val be '000002000000, which as such, is a positive number too large for 20 bits! (since the largest positive number for a 20 bit field is '000001000000). BIND will blow the whistle on an attempt to specify negative numbers that don't have the sign extended all the way out to 36 bits, since it has no way of knowing what we intend. So the correct octal value for -1 in a 20 bit field would be '777776000000, which is more conveniently accomplished by saying something like -'2000000. Got that?

BIND, like GET and GIVE complains about not understanding element and field names when it encounters garbage by invoking the error handler.

25.2. BIND_FIELD

External simple integer procedure BIND_FIELD(integer id, name, value, justification).

This acts just like BIND above, but it has an additional field which allows you to temporarily (for this call only) change the packing mode. For example,

```
BIND_FIELD(mod3,COEFF0,'1777777,LEFT_JUSTIFIED)
```

will select that packing mode for loading COEFF0 this time.

25.3. SET_MODE

External simple procedure SET_MODE(integer name).

This handles all the instances where the assembled word can be specified with only one argument. This includes some (but not all) CONO-A instructions, the MISC commands, and some pseudo-ops. (Pseudo-ops are interpreted by the assembler as instructions to the assembler to reconfigure itself in some way, rather than as an instruction to assemble a command. Since they look like any other request to assemble a command, they are given the prefix "pseudo".)

If one uses this to assemble CONO-A instructions, one must be sure to have have a CODE_STREAM channel set with a call to SET_OUTPUT (see below).

Class	Operation	Description

CONO-A	RESET_TICK_COUNTER	to beginning of pass (if stopped).
	PERMIT_PROCESSING_TICKS	
	INHIBIT_PROCESSING_TICKS	only update ticks are executed.
MISC	RESET	master reset.
	WAIT_CLEAR	start up all waiting process elements.
	PAUSE_CLEAR	start up all pausing process elements.
PSEUDO-OP	STOP	(screeech...)
	OPTIMIZE	combine adjacent combinable commands.
	NON_OPTIMIZE	pack one command per word,

Fig. 19

Example: SET_MODE(PAUSE_CLEAR).

25.4. SET_FIELD

External simple procedure SET_FIELD(integer name,field).

This is used to assemble code requiring two arguments, including some more CONO-A instructions, the TICKS and TIMER commands, and a couple of other pseudo-ops.

Class	Operation	Description
CONO-A	CONTROL_MODE	called only for three definitions below.
	C_START	start the box.
	C_TICK	cause one tick.
	C_STOP	stop the box. This command is identical to STOP above, except that this one is for use by the controlling system.
TICKS	PROCESSING_TICKS	set how many processing ticks per pass.
	TOTAL_TICKS	set how many of both kinds of ticks per pass.
TIMER	LINGER	process no more update ticks until pass counter = data.
	SET_PASSES	set pass counter to data.
	CLEAR_PASSES_DWELL	clear pass counter then LINGER until pass
		number equals field argument.
PSEUDO-OPS	SIZE_BUFFER	controls size of command buffer. Initialized to 1024.
	SIZE_COMMANDS	Sets how many commands will be sent to the buffer at once. Initialized to 64.
	PACKING_MODE	called only for three definitions below:
	RIGHT_JUSTIFIED	take low order 20 bits of source, pack into low order end of box destination field
	LEFT_JUSTIFIED	take low order 20 bits of source, pack to high order end of box destination.
	FULL_WORD	low order to low order, full width.

Fig. 20**Example:**

```
SET_FIELD(PROCESSING_TICKS, 300); (Destination will be COMMAND_STREAM.)
SET_FIELD(LINGER, CLOCK_RATE*5);
SET_FIELD(PACKING_MODE, FULL_WORD); (Pseudo-op, no code is assembled.)
SET_FIELD(CONTROL_MODE, C_START); (Destination will be CODE_STREAM.)
```

25.5. Assembler tick and pass counters

The next two variables keep track of the current update tick, and the current pass number. When the user assembles TOTAL_TICKS and PROCESSING_TICKS commands, the assembler stores away these values internally as well. For every command assembled, it then bumps the UPDATE_TICK counter by one, and when it exceeds the update ticks per pass, it increments the PASS counter. In addition, when a SET_FIELD call assembles a command of class TIMER, the PASS counter is reset to the appropriate value (either set from the data for LINGER and SET_PASSES, or cleared for CLEAR_PASSES). The SET_FIELD call to a TIMER class command also clears UPDATE_TICK, since a LINGER of any kind causes no

further update commands to be processed until the first update tick of the named pass, which will be update tick 0.

External Integer UPDATE_TICK.

External Integer PASS.

Section 26

Stream output

These routines deal with initializing the assembler's user tables, handling I/O, error trapping and other things.

26.1. INITIALIZE

External simple procedure INITIALIZE.

This has the following effects:

1) clears the unit_generators array; 2) resets the output streams; 3) clears the pass and tick counters. Initialize is done automatically when the first call is made to any of the low level routines. It can be invoked explicitly by calling the procedure INITIALIZE.

26.2. FLUSH

External simple procedure FLUSH.

FLUSH should be called before exiting the user program. It forces data left in all buffers to be written out. Failure to do so results in missing words at the end. So please, FLUSH when you're through... and wash your hands.

26.3. SET_OUTPUT

External Simple Procedure SET_OUTPUT(Integer stream_type,sail_channel).

SET_OUTPUT specifies to the assembler which CPU I/O channel to use to write out the assembled code, for instance to a file-storage type device such as a disk or tape, or directly to the Synthesizer.

SET_OUTPUT does not do any calls to the operating system to set up such a channel, it's sole function is to tell the assembler about a channel which has been opened for it. So the actual sequence of use would be to do calls to the CPU monitor to open the appropriate channel, then to pass the channel number so opened to the SET_OUTPUT routine.

As the assembler can be used to make up either/both commands and/or instructions, and as these are usually kept separately, one must specify different destination channels. This is done by setting the Integer stream_type to `COMMAND_STREAM` for commands, or `CODE_STREAM` for instructions. (It is possible to merge both types by giving the same channel in two calls to `SET_OUTPUT`.)

In addition, at Stanford, up to 16 channels of output are available to any job, so that the same stream can be output to multiple channels up to that limit.

26.4. SET_PROCEDURE

External simple procedure `SET_PROCEDURE(reference procedure <name>)`.

`SET_PROCEDURE` allows the user to choose a home grown procedure for error recovery. When an error is detected by `LOWER`, the error handler first stops execution then, assuming you have declared an error procedure of your own with `SET_PROCEDURE`, it passes control to your procedure and passes a `SAIL` type string to it containing a thumbnail characterization of your problem. What you probably want your error procedure to do is to save the current state of the world, close files that were open and generally make a controlled retreat.

It should be possible to continue execution from most errors except those having to do with I/O. In the instances where illegal processing element identifiers are encountered, or data fields are too big, and suchlike, continuing causes the assembler to not assemble that word. An instance of using `SET_PROCEDURE`:

```
Simple Procedure box_error(Reference String errmsg);
Begin
  Print("You have done the following no-no:"
        "&errmsg&'15&'12");
  Close(input_file);
  Close(output_file);
  Release(input_file);
  Release(output_file);
  Usererr(0,0,"Low level error number 22");
End;
(at some point during initialization of the user program, you then say...)
SET_PROCEDURE(box_error);
```

Section 27

Disassembly - Reassembly

Two programs to aid debugging are available, a command disassembler, DAB (...stands for DisAssemble the Box) and a command un-disassembler, called UNDAB (got that?).

27.1. DAB

This program takes a file full of compiled Synthesizer commands and turns it into a text description of what each command means. Here is a sample output of DAB. The first column is the original command printed in octal; next column is the command number relative to the beginning of the file in octal; next is the processing element index (where appropriate, miscellaneous commands have no processing element number); next comes the command op-code, printed as the letter G, M or D plus "." plus field mnemonic (example: G.MODE); then comes the 20 bit data field; and lastly a terse description is given of what it all means. The description is sometimes accompanied by the value of the datum expressed as a fraction.

2170600;	0	HiProc	217	;Highest Proc Tick #
3060610;	1	HiTick	306	;Highest Tick # - 1 (25641/sec)
100;	2	TTLa	0	;Load TTL register A
10140;	3	TTLb	1	;Load TTL-B;Filter 9000
10430;	4	ClrP	1	;Clear Pass Counter;Linger
401025000;	5	0 G.MODE	200	;Mode dac l-q sin
;		0 G.FM	102	;Fm address
5400;	6	0 G.O	0	;Sweep rate or DAC #
10000006400;	7	0 M.M0 l	1000000	;Coefficient 0 w/DX = .500000
6200;	10	0 M.M1 r	0	;Coefficient 1 = .000000
400007600;	11	0 M.IN	0	;A input sum address
;		0 M.RM	100	;B input sum address
1210027400;	12	0 M.MODE	24	;Mode mixing
;		0 M.SCALE	4	;Scale M0*2 M1*1, int M0/2 M1/4
;		0 M.SUM	2	;Output sum address
401035001;	13	1 G.MODE	200	;Mode dac l-q sin
;		1 G.FM	103	;Fm address
15401;	14	1 G.O	1	;Sweep rate or DAC #
10000006401;	15	1 M.M0 l	1000000	;Coefficient 0 w/DX = .500000
6201;	16	1 M.M1 r	0	;Coefficient 1 = .000000
37754273004;	77	4 G.P	3775427	;Decay rate
12010420;	100	Linger	1201	;Absolute Linger
0;	131	Noop		;Misc - No effect
12000007403;	130	3 M.MODE	0	;Mode inactive

The program is SYS:DAB.DMP at Stanford. When run it asks for an input file which is presumed to be full of Synthesizer commands and then asks for an output destination. If "TTY:" is given as the input file, then DAB will ask for direct TTY input. Filenames will have a default

input filename of TEST.SAM and output filename of TEST.DAB. The program supports partial filename specification. The filename can be entered on the command line after a semicolon (e.g., .R DAB;FOO<return>).

27.2. UNDAB

UNDAB takes a file of disassembled commands (a text file) such as shown above and turns it back into an assembled one (back into binary).

It is frequently useful when developing programs to generate command streams for the Synthesizer to be able to change a compiled command stream by hand, then reassemble it directly. This is easily done using DAB and UNDAB. One can modify the commands in the .DAB file using a text editor of the user's choice, then reconvert the file to binary with UNDAB. UNDAB only reads the command part of the disassembly, and ignores everything between the semicolon and the end of the line.

27.3. MERGE

This program takes discrete command files and merges them together, bumping all the processing element numbers around to make it all fit. Nobody has ever used it, so it most likely does not work too well.

Section 28

How to crank up the Samson box

There are some important things to realize about the order in which things are done to crank up and run the box. Here's a general discussion of things to do, with a particular discussion below of exactly how to "cold start" the box.

28.1. General considerations

You should set or clear all relevant fields in the box before you crank it up. Any processing chains should be started up in the order that they reference each other. Also, if modifiers are reading from this pass sum memory, the writing processing element must have done its thing before the reading modifier. So a typical program will do the following in roughly this order:

- Require LOWER.DEF source_file and LOWER.REL load_module,
- Do a SET_OUTPUT to pass a command channel to the assembler,
- Calculate the number of ticks needed, pass that with SET_FIELD to the assembler,
- Set the corresponding sample rate, and magic numbers,
- GET the desired processing elements,
- Including generators to output samples to the DAC, if desired
- Set all unused processing element fields to reasonable values with BIND,
- Set any unused FM ports in generators or A_IN type ports in modifiers to some unused (hence 0) sum memory location with GET and BIND,
- Set all used fields to their desired values with BIND,
- Connect processing elements to sum memory (and hence to each other),
- Start the processing elements in order of reference
- LINGER until the event requires more update ticks,
- or if it's finished, GIVE back the used process elements and sum memory locations when no longer needed so other processors can get high-ordered time slots.
- lastly, FLUSH!

28.2. Cold starting the box

What I am about to describe is a mysterious, arcane ritual that has evolved over time into its present form. Not all elements of the incantation have ready explanations, their reasons being

buried in the mists of antiquity. Where we have explanations, they are often complicated. But, so far as we know, performing these rites PRECISELY as stated will achieve the desired effect.

The first phase of cranking up the box requires cranking it down -- taking it from an arbitrary state, unknown by us, to a known quiescent state. In our literature, this is referred to as a STIFLE. Fortunately, the SIX operating system knows how to do this if you send a MSTIFL message to the SAM device. But should you need to know how to do it yourself, this is what a STIFLE currently involves:

Master reset - Has a variety of effects, as described in the [Specification].

Start clock; Stop clock - This is necessary so that the effects of the master reset can propagate thru internal pipelines. It only requires about 8 clock ticks to do this, so this gives plenty of time without using 8 clock tick commands.

Clear DX, wait and pause bits - To avoid surprises.

Clear generator parameters - To achieve known state, and to avoid generators wanting read and write data WCMA's. Includes clearing modes.

Clear modifier parameters - To achieve known state, and so delay clearing doesn't have interference. Modes are parameters.

Clear delay memory - To avoid nasty noises in the reverberation, we want to zero all of delay memory. But this must be done by running the box, since there are internal buffers that can only be cleared in this fashion. All 32 delay ports are exercised by running 32 modifiers in parallel (which is also the fastest way to clear memory), lingering until enough time has passed to have written zero's everywhere, then halting. The halt can be detected from the computer with a CONI-A.

Clear parameters used by delay clear - We want everything cleaned up.

Reset ME, PE and NX errors - Writing delay memory may have detected parity errors (especially if we just powered up the box), and we don't want to see meaningless errors.

Now that the box is completely shut down, we can begin to crank it up with predictable (it says here) results. Please note that the steps that follow generally cannot be sidestepped or combined, regardless of appearances. With the clock stopped, begin the following sequence:

Set number of total and processing ticks - This is best done before starting the clock. If this can't be done, consult a wizard, since the box can find itself in various weird states depending on the previous and new settings of these values. Lacking exact information, use 1 processing tick and '1775 total ticks here.

Set read data packing mode - Need to do this before doing any read data, but can be omitted otherwise.

Permit processing ticks - So sound will come out.

Reset tick counter

Load tables into delay memory - Omit if no tables are used.

DATE!

How to crank up the Samson box

DRAFT

Load WCMA's for commands, read and write data - Only the WCMA's for data streams actually used must be set of course.

Enable stop/interrupt for desired error conditions

Start clock - That's it; box is off and running.

28.3. Cold starting from a real-time job

This is a sample of code for a Real Time Job on the PDP-6, the CPU which is currently used to control the Synthesizer. SNDMES and other non-Tops-10 instructions are UUOs to the PDP-6's monitor. For details on the Stanford system, see [Harvey].

```

SNDMES [FNOWDS!MSTIFL,,SAMDEV]      ; DO A STIFLE
SNDMES [MDTA,,SAMDEV                ; SET HI PROC TICK NO. TO 1
      1,,0
      10600]
SNDMES [MCONOA,,SAMDEV              ; TICK THE CLOCK ONCE
      1,,0
      600000]
SNDMES [MDTA,,SAMDEV                ; SET HI TICK NO. LESS 1 TO 1
      1,,0
      17750610]
SNDMES [MCONOA,,SAMDEV              ; TICK THE CLOCK ONCE
      1,,0
      600000]
SNDMES [MCONOB,,SAMDEV              ; SET READ DATA PACKING MODE
      1,,0
      7]
SNDMES [MCONOA,,SAMDEV              ; THIS IS FOR 16-BIT, USE 27 FOR 20-BIT
      1,,0
      4000]
SNDMES [MCONOA,,SAMDEV              ; RESET TICK COUNTER
      1,,0
      100000]
SNDMES [MDMWR,,SAMDEV               ; WRITE DELAY MEMORY TABLES
      <N+1>,,0
      <-N>,,<delay memory address>
      ... N 20-bit data words ...]
SNDMES [MRDBUF,,SAMDEV              ; QUEUE READ DATA BUFFERS
      1,,0
      <-N>,,RDBUF0]
SNDMES [MRDBUF,,SAMDEV              ;
      1,,0
      <-N>,,RDBUF1]
SNDMES [MRDBUF,,SAMDEV              ;
      1,,0
      <-N>,,RDBUF2]
SNDMES [MWDBUF,,SAMDEV              ; QUEUE WRITE DATA BUFFERS
      1,,0
      <-N>,,WDBUF0]
SNDMES [MWDBUF,,SAMDEV              ;
      1,,0
      <-N>,,WDBUF1]
SNDMES [MWDBUF,,SAMDEV              ;
      1,,0
      <-N>,,WDBUF2]
SNDMES [MCBUF,,SAMDEV               ; QUEUE COMMAND BUFFERS
      1,,0
      <-N>,,CBUF0]
SNDMES [MCBUF,,SAMDEV              ;
      1,,0

```


DATE!

How to crank up the Samson box

DRAFT

```
<-N>,,CBUF1]
SNDMES [MCBUF,,SAMDEV
1,,0
<-N>,,CBUF2]
SNDMES [MCONOB,,SAMDEV          ; ENABLE STOP/INTERRUPT - 1 MSG PER ENABLE
1,,0
<consult specs for appropriate values>]
SNDMES [MCONOA,,SAMDEV          ; START THE CLOCK, AND AWAY IT GOES
1,,0
400000]
```

28.4. Sample code for a SAIL program on the KL-10

This example, written in SAIL does the same thing as the example above, likewise written by Ken Shoemake. The MESOU_n procedures are from a library of such routines for SAIL consumption, the effect is as in the examples above.

```

PROCEDURE SConoA(
    INTEGER datum);
COMMENT Sends MCONOA message to the SAM device on the SIX.;
BEGIN "Send MCONOA"
    INTEGER seq;
    MesOu3(MCONOA, samdev, seq, RightHalf(datum));
END "Send MCONOA";

PROCEDURE SConoB(
    INTEGER datum);
COMMENT Sends MCONOB message to the SAM device on the SIX.;
BEGIN "Send MCONOB"
    INTEGER seq;
    mesou3(MCONOB, samdev, seq, RightHalf(datum));
END "Send MCONOB";

PROCEDURE SDataoA(
    INTEGER datum, TickIt(TRUE));
COMMENT Sends MDTA message to the SAM device on the SIX,
    then ticks the clock once if TickIt is TRUE.;
BEGIN "Send MDTA"
    INTEGER seq;
    MesOu3(MDTA, samdev, seq, datum);
    IF TickIt
        THEN SConoA('660000);
    END "Send MDTA";

MesOu1(MSTIFL, samDev);      "Stop whatever box was doing before"

SDataoA('10600);      "Set hi proc tick no. to 1 if unknown"
SDataoA('17750610);   "Set hi tick no. to 1775 if unknown"
IF Packed
    THEN SConoB('7)    "Setup Read Data packing mode"
    ELSE SConoB('27);   "This is 16-bit read data"
    ELSE SConoB('27);   "This is 32-bit read data (of which 20 bits are used)"
SConoA('4000);         "Permit processing ticks"
SConoA('100000);       "Reset tick counter"

"Queue up read data buffers"
Dsk6Rd(RdChan, RdSize, RdBuf0); MesOu3(MRDBUF, samdev, seq, MakeWcma(RdSize, RdBuf0));
Dsk6Rd(RdChan, RdSize, RdBuf1); MesOu3(MRDBUF, samdev, seq, MakeWcma(RdSize, RdBuf1));
Dsk6Rd(RdChan, RdSize, RdBuf2); MesOu3(MRDBUF, samdev, seq, MakeWcma(RdSize, RdBuf2));

"Queue up write data buffers"
Dsk6Rd(WdChan, WdSize, WdBuf0); MesOu3(MWDBUF, samdev, seq, MakeWcma(WdSize, WdBuf0));
Dsk6Rd(WdChan, WdSize, WdBuf1); MesOu3(MWDBUF, samdev, seq, MakeWcma(WdSize, WdBuf1));
Dsk6Rd(WdChan, WdSize, WdBuf2); MesOu3(MWDBUF, samdev, seq, MakeWcma(WdSize, WdBuf2));

"Queue up command buffers"
Dsk6Rd(CChan, CSize, CBuf0); MesOu3(MCBUF, samdev, seq, MakeWcma(CSize, CBuf0));
Dsk6Rd(CChan, CSize, CBuf1); MesOu3(MCBUF, samdev, seq, MakeWcma(CSize, CBuf1));

```

```
Dsk6Rd(CCChan,CSize,CBuf2); MesOu3(MCBUF,samdev,seq,MakeWcma(CSize,CBuf2));
```

```
"Enable stop/interrupt on various conditions, 1 msg per enable"  
SConoB("consult specs for appropriate values");
```

```
SConoA('400000);    "Start clock"
```

Appendix 1

An introduction to pipelining

There are a number of ways to accomplish digital sound synthesis. If we used a general purpose computer to do the processing for us, depending on which kind we used and how busy it was otherwise, we might be able to accomplish our goal, but there would be lots of wasted motion. General purpose computers are meant to be able to handle a wide variety of kinds of jobs. Consequently, there is lots of extra baggage that they carry along to allow this flexibility. We, however, have a rather specific job to do: most signal processing jobs involve applying a long series of data to a relatively fixed complex calculation. So the first thing to do if we were designing a computer for our purposes is to dump as much of the overhead and waste motion as possible (however, it would be nice if the device we were to build were controlled by a large general purpose computer so that we could have as flexible a way as possible to run it).

Let us take the following equation as an example of a calculation we might want to do with our machine. It produces a cosine wave when fed lots of increasing values of I,

$$Y \leftarrow A * \sin(6.28138 * (I/512) + \pi/2).$$
Equ. 21

Our calculator will only need to be able to add, to multiply and divide and to look up values in a sine table. It would do the calculations in stages, just as we would by hand: it would form intermediate results and then combine them and put out the result. Such a machine, without the overhead of a general computer, could be made to be very fast.

But this still has some problems. What if the formula we want were still too time consuming to calculate at the rate of speed we want? What if we wanted lots of these formulas to be running simultaneously and added together? How do we expand this basic process but still keep the average computation time down?

If we just got lots of these processors and fed each one a number we would cut the average compute time by 1/<the number of extra processors>. But we'd need some fancy system of passing the numbers out and collecting the results.

A better approach is to realize that, since the calculation is broken down by the processor into intermediate steps, it might be possible to have one processor working on several values of I at once. To see how this can happen, we must know the difference between block structure and pipelining. We have been unconsciously treating the processing of the formula above in block structure. In this mode, we take a number for I, subject it to all the intermediate computations and wait for a final value for Y to pop out before we take another value for I. But what if we did it this way:

Lets make an "assembly line" out of the process. A belt carries in the values of I, lets say we have

5 values. The workers along the belt are the divider, the multiplier, the adder and the sine-table-looker-upper, standing in the order that we would normally solve the equation. Here comes the first I . The divider picks up his number cruncher and divides it by 512. Meanwhile the rest of the guys multiply, add and sine-table-lookup whatever junk the janitor left on the belt last night, maybe his lunch or something. They all place their results down on the belt, and it is carried along to the next one. The divider's number is now in front of the guy who multiplies by 2π . Meanwhile, THE NEXT VALUE OF I comes down the chute towards the divider! Notice we don't have a valid value for Y at the other end yet. But if we continue, we will. So, the divider and the 2π multiplier get to work doing their thing, meanwhile, the rest of the guys continue playing with the janitor's lunch as it moves past them on the belt, rolling it into little balls and throwing it at each other. They set down their results, and now the adder gets the intermediate value of what was the first I , the 2π multiplier gets the next, and the divider gets yet another new I . On the next shift, the sine-table-looker-upper does his thing, and the value for the first I is through the process, transformed into Y . Behind it are the other intermediate values. So we need to crank the whole process three more times to get the 5th Y value through the line. But notice, during these next three passes, we get a valid Y value every time the assembly line turns around once. That means we get a good result in the time of only one intermediate step, whereas in the block process, we would have got only one result for every five intermediate steps. This is a nice increase in average computing time. It's called pipelining.

Lets look at some of the implications of this process. First of all, all the inner operations are similar in that they constitute the picking up of a number, the operation on it, and replacing it. We'll call this basic task a pass. Furthermore, it should be obvious that the system cannot work any faster than the slowest processor. The minimum execution time of a pass then is the time it takes for the slowest computing element. So the accomplishment of one intermediate result consists of a pass.

Notice that the processor must be primed like a pump., but once it is, it produces one good value of output every pass. In general, if the pipeline is N elements long, we must add N passes to the number of passes it would take to send all the numbers through the pipeline.

Notice also we must throw away the first N samples we get of the output. In the assembly line above, the processor was four units long, and we had five values of I to calculate, so it took nine passes to accomplish the task, and we threw away the first four values of the output since they were undefined (unless you knew the janitor).

Appendix 2

Time division multiplexing

The Samson box is extensively pipelined. Just how extensively we will soon see. Two of the essential ingredients in the box are called processing elements. One of them is called a modifier, the other is called a generator. The modifier does various flavors of arithmetic and logic testing, while the generator does a fancier version of our example formula, but with lots of bells and whistles. Pete Samson claims there are 256 generators and 128 modifiers in his box. In fact, there's only one nest of electronics for the generators and one for the modifiers. What really goes on is that there are 256 sets of parameters for the generator and 128 sets of parameters for the modifier, and the processing elements zap from one set to the next at a truly blinding rate of speed. This is called time division multiplexing.

To get the full picture, reload your mental image of the example of pipelining above. Think of it maybe this way:

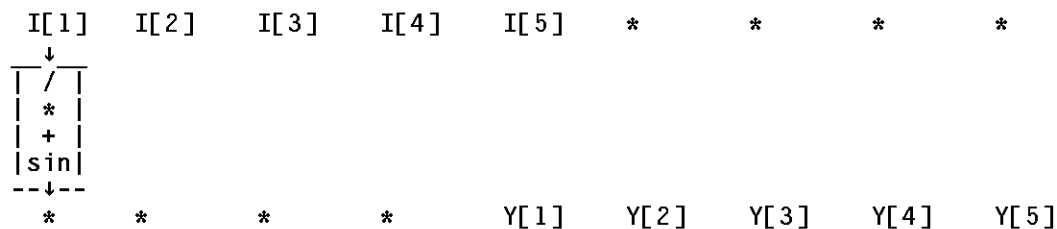


Fig. 21

Each step the box takes from left to right accomplishes one pass, and on the fifth pass, we start getting valid Y values. Now imagine the process if we want to deal with many streams of data. All we need do is interleave the separate streams. Input I[1] will come out as output X[1], J will come out as Y, and K as Z.

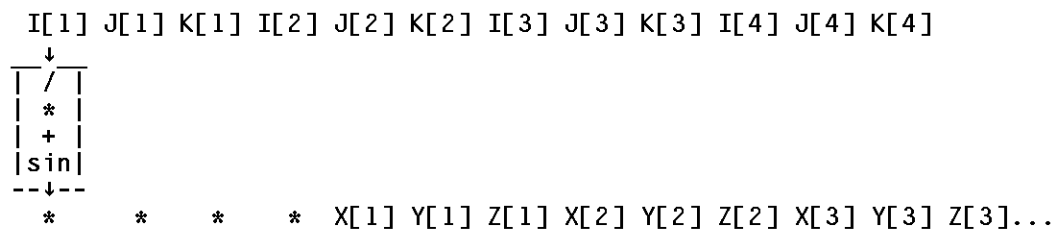


Fig. 22

Before analyzing this, let's redefine our terms. What we were calling a pass, let's now call a tick. So a tick is the act of moving the processor, sucking up a number and spitting out another. Now let's redefine "pass" to mean that point where we have produced one valid output for all of the streams. (Don't worry, we'll stick to this definition. The term pass can be used in simple situations like our first example, but from here on we reserve the word pass to refer to the period culminating in the output of samples.) So, where is the end of the first pass? Right!, at Z[1], after seven ticks. Where is the end of the next pass? Right again, at Z[2], after three ticks. From here on, a pass consists of three ticks only. Since one pass equals one valid output, if we have five each of I, J and K, it will take five passes of three ticks per pass to get them all through PLUS four ticks to prime the processor for a grand total of 19 ticks.

We can deduce from this that the number of ticks per pass is equal to the number of parallel data streams, also that the total number of ticks is equal to:

$$<\# \text{ of streams}> * <\text{length of longest stream}> + <\text{length of pipeline}>.$$

Equ. 22

Remember that a pass consists of all the processing necessary to produce one complete set of output data, that is, one sample. The tick is, therefore, a subspecies of the pass. The tick is the smallest unit of time our process can use and the time it takes is the time of the slowest process.

Appendix 3

Basic number theory

Trying to make representations between different number systems is somewhat the same problem as finding suitable translations for unidiomatic foreign terms, we have to agree on a meaning.

Modern computers represent numbers in a binary system. Briefly what a binary number consists of in such a machine is a group of electronic devices each of which can be "on" or "off", depending on the number being represented. Each device stands for a particular weight or magnitude, and the number represented by the set of devices is the weighted sum of the ones that are "on". It is exactly the same thing to think of weighing an object on a ballance scale using individual weights of different magnitude.

Now if we let three printing columns represent three such devices and we wanted to represent the number 3 we would do this:

weight:	4	2	1
state:	0	1	1

Fig. 23

where a 1 means "on" and 0 means "off". The word binary refers to the fact that the devices have only these two states. The general expression for positive integer values of such a number is (don't panic)

$$b_m 2^m + \dots + b_2 2^2 + b_1 2^1 + b_0 2^0 \wedge$$

where the " $\wedge$ " is the "binary point" (think of decimal point). The b term is the state, m is the largest weight. The b term can be 0 or 1, depending if the number is "on" or not. The subscript m counts which weight we are discussing. Weight number m has a magnitude of 2^m . In terms of our example of the value 3 above, this would be:

$$\dots 0 2^2 + 1 2^1 + 1 2^0 \wedge$$

The largest unsigned integer we could represent would be $2^m - 1$ where m is the number of places, or "bits" left of the binary point. In Fig. 23, 7 is the largest number we could represent, starting from zero.

Fractions are expressed as:

$$\wedge b_{-1}2^{-1}+b_{-2}2^{-2}+...b_{-n}2^{-n}$$

where n is the smallest magnitude. For example, signed binary fraction 0.11 (decimal +.75) is:

$$0_{-1}2^{-1}+1_{-2}2^{-2}+1_{-3}2^{-3}$$

For representing the domain of positive and negative numbers there are three basic systems used: sign-magnitude, one's complement and two's complement. In the sign-magnitude system the leftmost bit, called the most significant bit (MSB), or high order bit, is used to denote the sign of the number, zero being taken as positive, one as negative. So the highest integer magnitude that can be represented is $2^{n-1}-1$.

One's complement and two's complement do things a little differently. Here the negative numbers are expressed as the logical complement of their positive counterpart. Complementation is done by changing all the zeros to ones and vice versa. For one's complement the formula for the negative equivalent is 2^n-x where n is the number of bits and x is the number to be complemented. So the one's complement of 001 is 110, which we represent in decimal as +1 and -1 respectively. For two's complement the formula is $(2^n-1)-x$ or $(2^n-x)-1$. For example, to form the two's complement of 010, subtract one = 001, and complement = 110. Or, equivalently, complement 010 = 101 and add one to the least significant bit (or LSB) = 110. To show this graphically:

binary	two's	one's	sign-magnitude
011	+3	+3	+3
010	+2	+2	+2
001	+1	+1	+1
000	+0	+0	+0
111	-1	-0	-3
110	-2	-1	-2
101	-3	-2	-1
100	-4	-3	-0

Notice that the one's complement and sign-magnitude have two representations for zero, whereas the two's complement has one zero and it is taken as positive. Two's complement also has one negative number greater than its largest positive number. This number has no complement. Notice that the two's and one's complement system "wrap around" from the greatest positive number to the greatest negative number when the greatest positive number is incremented whereas the sign-magnitude system goes to zero again.

Now the Samson box uses two systems, the two's complement for signed numbers and plain binary for unsigned numbers. Two's complement offers some advantages to the computer designer and is nearly universal (except for some very old dinasaures!).

So far so good, now what if we wanted to represent fractional numbers instead of integers? Well, for example, using the unsigned system, and expanding to four bits it might look something like this:

binary fraction	decimal fraction
.1111	nearly 1.0
.1110	
.1101	
.1100	.75
.1011	nearly .75
.1010	
.1001	
.1000	.5
.0111	nearly .5
.0110	
.0101	
.0100	.25
.0011	nearly .25
.0010	
.0001	
.0000	.0

The fractional value is then the $\langle \text{decimal value} \rangle / 2^n$.

Notice that we can't represent exactly 1.0 with this notation. The value for exactly 1.0 would be 1.0000, but we are purposely limiting ourselves to 4 bits, so we can't do it. (If you have only one decimal digit you can represent 0 9 ok but will have a little trouble with 10!) This value of "nearly 1.0" is as close as we can come so long as we limit ourselves to defining all the bits to be part of the fraction. Whenever the value of "exactly 1.0" is wanted, the value of "nearly 1.0" must be substituted. There is a special expression for this situation which requires the introduction of another symbol: ϵ . This symbol stands for the smallest positive value in any number representation. In the case above it has the value of .0001. This is called the "epsilon value". We can now express "nearly 1.0" as $1 - \epsilon$, which it is: $1.0000 - .0001 = .1111 = \text{"nearly 1.0"}$.

The two's complement version of fractional binary is:

binry	fractional
0111	1- ϵ
0110	
0101	
0100	.5
0011	.5- ϵ
0010	.25
0001	.125
0000	.0
1111	ϵ
1110	-.125
1101	-.25
1100	-.5
1011	
1010	
1001	-1.0+ ϵ
1000	-1.0

Here we have a -1.0, but only a +1- ϵ or "nearly +1.0". So this conforms with the integer version of the two's complement system which also has one greater negative value than positive.

This numerical system has the formal name of two's complement fixed binary point notation (quite a handle!). It is the primary number representation in the Synthesizer and is used everywhere that a signed number is needed, most notably for waveforms, which are representations of positive and negative fluxuations around normal air pressure taken as zero. However, it is not used in simple counting situations such as the ANGLE term, where the unsigned fractional notation is used.

There are some useful properties in binary number systems to be aware of: Notice the sequence in the fractional notations that .5 = 0100..., .25 = 0010... and .0125 = 0001... (where "..." stands for any additional bits to the right). From this it can be seen that to multiply a number by a (positive) power of two, N, requires only that it be shifted left N places. For division by a power of two, shift right. This also holds for integer binary systems: ...0010 = 2^2 , ...0100 = 2^4 , etc.

Be careful as to whether a number is signed or unsigned when combining them in expressions as signed numbers are right shifted one bit compared to unsigned numbers (which use the high order bit for magnitude, not sign).

This also brings up the general problem of adding words of different lengths. Clearly, to get a valid result for an addition or subtraction, the weights of the terms must match. If the words are of different lengths, the procedure is to first normalize the two numbers (line up the weights), then extend the sign bit of the shorter word out to the limit of the most significant bit, then add.

Appendix 4

More about processing speed vs. capacity

4-1. Speed of processing

The total number of ticks per pass is set by the programmer. This is so because there is a payoff between passes and the two kinds of ticks. The nut of the problem is that, when all is said and done, the basic speed of processing one tick is 195 nsec. (That's .0000000195 sec., or 1.95×10^{-7} , or 195 billionths of a second.) That's the time that it takes a processing element to do the slowest thing it has to do. So the whole box is geared to go at this rate by an internal clock. This means that the box does 5,128,205.128 ticks per second.

Now, as you recall, the whole purpose of the box is to produce bunches of numbers which are passed at blinding speeds to the DAC. Each number sent to the DAC represents a complex calculation of the value of whatever waveform the box is producing. Taken through time, the individual samples sent to the DAC produce the desired waveform. The samples must be passed to the DAC at a uniform rate, and at a sufficiently high speed so as to get a good high frequency response. For instance, a typical sampling rate of 12800 samples per second produces a usable frequency response from 0 to 6400 Hz. Faster rates produce higher frequency responses. So naturally we want a good fast sampling rate. But we've got to watch out for a catch, namely, that the box can only do so much so fast. The barrier is the 195 nsec. tick turn around time.

4-2. Ticks per pass

The problem should become clear when we try to feed samples to the DAC in real time. Real time, in this example means that we will attempt to guarantee that the box will send 12800 samples to the DAC per second at a uniform rate. That means one pass takes 7.8125×10^{-5} seconds. Furthermore, it must send them at an even rate, such that each pass gets computed and each sample shipped out in $1/12800$ th of a second. How many ticks can we squeeze out of this much time? That comes to $7.8125 \times 10^{-5} / 1.95 \times 10^{-7} = 400$ ticks per pass. This then is the maximum number of ticks of either kind that can be squeezed into a pass. That's a pretty good number, actually. That means we could have all the generators running (and half the modifiers since they run concurrently) using up 256 ticks and still have 144 left as update ticks.

However, this is not a very fast sampling rate, and gives us an upper frequency response limit of 6400 Hz. If we wished to up this rate, we'd have to decide whether we wanted to have fewer processing or update ticks, since the pass period is their sum (plus overhead ticks which are invariable).

4-3. Processing vs. update ticks

Not only is there a payoff between the desire to have the sampling rate as fast as possible and the desire to have as much processing as possible per pass, but there is a payoff between the two kinds of ticks, to wit: Processing ticks and update ticks occupy mutually conflicting time domains. All the processing ticks are performed first, then what's left is given to the update ticks. So if your piece requires lots of real time user interaction with keyboards or what have you, or if you are repatching the connections and modes of the processing elements all the time, then you will be forced to allocate a larger part of each pass to sending update information to the box, and that means lots of update ticks per pass. And since no processing can be done in a processing element while it is having its parameters updated, this reduces the number of processing ticks available on any given sample, and means that the instrument must be simpler than one which had fewer update ticks. The obverse works as well: too complex an instrument means you have fewer update ticks per sample. So, as old Richard Almaniack used to say, "budget your ticks wisely!"

4-4. Numbering of elements and ticks

The processing elements are all numbered ordinally. Generators go from 0 to 255 modifiers go from 0 to 127. Processing always begins with the lowest numbered running processing element and goes sequentially until the end of the pass. If, for instance, you have 128 processing ticks, you can run all the generators up to #127, all the modifiers up to #63. The remainder of the processing elements will be inactive. Thus, the first tick is tick 0.

When actually assembling the value to be sent to the synthesizer to set the total ticks and processing ticks, the number to specify is the highest tick to be used, not how many ticks to use. That is, you want to give the "relative" tick number, not the "absolute" number of ticks. In the above example, to get 128 processing ticks, tell the box that the highest tick is #127.

There's one additional wrinkle for setting the total ticks. Here the box expects the next-to-highest-tick. For example, say we had 96 processing ticks plus 32 update ticks per pass for a total of 128 total ticks. The highest tick is #127, the next-to-highest-tick is therefore 126, and this is what you would tell the box to get 128 total ticks.

Appendix 5

SAIL examples

5-1. Generator example

What we want to do now is to set up some intermediate level routines out of the low level that will do such basic things as set up DAC channels, sampling rate, magic numbers, file output, etc. Then we'll call the procedures in the context of a couple of abbreviated programs. These procedures will be specific to our example programs; to make them truly general might obscure their didactic purpose.

Our first "piece" will be to generate 6 seconds of a sine wave at A 440. For you analog synthesizer cognoscenti, I've tried to write this example to make sense in terms of the equivalent patching procedure.

The first procedure will set up a generator and a sum memory location as a channel to a DAC. Assume from now on that ∂ stands for COMMENT.

```
integer procedure GET_OUTN(integer chan; reference integer gen_out);
     $\partial$  Gets DAC output generators;
begin
    integer sm;
    gen_out $\leftarrow$ get(id_generator);       $\partial$  gen_out now has addr. of 1st.
                                       free generator;
    sm $\leftarrow$ get(gen_sum);               $\partial$  sm gets addr. of 1st. free sum memory;
    bind(gen_out, fm, sm);              $\partial$  gen_out's fm input now reads contents
                                       of sm;
    bind(gen_out, sweep, chan);         $\partial$  DAC addr. is put into the sweep slot;
    bind(gen_out, mode, dac_write);     $\partial$  Start up generator feeding DAC;
    return(sm);                        $\partial$  this addr. will be used by generators
                                       desiring output to DAC;
end;
```

This procedure sets the clock rate from the number of processing and update ticks. Here srate is the sampling rate, frmag the magic number for scaling frequencies, dfrmag is for SWEEP and gpmag is for RATE. These are all global variables which are set here.

```
procedure SET_CLOCK(integer nptix, nutix);
begin
    set_field(processing_ticks, nptix);
    set_field(total_ticks, nptix+nutix);
    srate $\leftarrow$ 1/(0.000000195*(nptix+nutix+8));
    frmag $\leftarrow$ (1 lsh 28)/srate;  $\partial$  frmag is the scaling
        for FREQUENCY;
```

```

    dfrmag←frmag/srate; ∂ dfrmag is the scaling for SWEEP;
    gpmag←(1 lsh 24)/srate; ∂ gpmag is the scaling for RATE;
end;

```

This next procedure will set up a file to record the command stream. To do this, get a system channel, open a disk file on it, then pass the channel number to SET_OUTPUT, which will take any code and pass a copy of it to the file, as well as to the box. Getchan and open are predeclared SAIL procedures which return an available system channel and open it, respectively. Outstr prints a message on a teletype, inchwl inputs characters to a string from a terminal until it gets a line feed.

```

procedure FILOUT;
begin
  string s;
  com_chan←getchan;
  open(com_chan,"dsk",'17,0,0,0,eof,fail);
  do begin
    outstr("output file = ");
    enter(com_chan,s←inchwl,fail);
  end until not fail;
  set_output(com_chan); ∂ this passes the channel number
    to the box assembler;
end;

```

The next two routines will setup and cleanup the box for this example. Zero is a global integer which will contain the addr. of a sum memory location which we will never write into, and which will hence always return zeros. ¹ Outa is a global integer which will have a DAC addr. set in GET_OUTN above. Gen_outa is another global integer which will be the addr. of the generator in WRITE_DAC mode. The BIND command makes the output of the unit generator producing the sound go to outa, which is read by gen_outa and passed to the DAC. ²

```

procedure INIT;
begin
  INITIALIZE;          ∂ lower level initialization;
  zero←GET(gen_sum);
  SET_CLOCK(64,64);    ∂ will produce a 38.4kHz sampling rate;
  outa←GET_OUTN(1,gen_outa); ∂ a generator now desiring to pass
    samples to DAC 1 need only put the samples in sum memory in
    location OUTA;
  FILOUT;
end;

```

¹ Since processing elements read sum memory whether we like it or not, if we don't want junk coming into it we must set it's input to read from zero.

² Note, however that since modifiers can't write into generator sum memory, a separate path for modifiers to the dacs must be supplied. See the solution in [Moorer].

This next procedure will clear the box for this example. Giving back these elements is not really necessary at this point since our "piece" ends here, but in principle whenever you aren't using a processing element, GIVE it back so subsequent GETs don't continually gobble up processing elements.

```

procedure FINISH;
begin
  GIVE(gen_outa);
  GIVE(outa);
  FLUSH; ∂ don't forget to wash your hands;
  close(com_chan);
  release(com_chan);
end;

```

These next two procedures stuff values into a generator. We only want to control the frequency and amplitude without anything else fancy in this example, but we must zero the things we don't want. We'll break it down into two procedures, one for frequency information, another for amplitude and enveloping.

```

procedure SET_GEN(integer ug; real freq);
begin
  BIND(ug, sweep, 0);      ∂ set up oscillator params., this one for
                           sweep rate = 0;
  BIND(ug, angle, 0);      ∂ initial angle = 0, so we get a sine wave.
                           Had it been (2↑20)/4, it would
                           have been a cosine wave, since
                           it would start at 90 degrees;
  BIND(ug, fm, zero);      ∂ the FM term is always read,
                           so if not wanted point it into
                           sum memory where it will only
                           get zeros;
  BIND(ug, scale, 0);      ∂ scale must be set to zero when not using
                           sum of cosines mode;
  BIND(ug, frequency, frmag*freq); ∂ set frequency;
  BIND(ug, sum_memory, outa); ∂ pass it's output to outa;
end;

```

This next will setup or update a generator's envelope side, and start the generator. We'll use an envelope shape that has a positive going attack from zero amplitude to 1. Gpmag*duraction gives an increment size that will step through these values in that time. The steady state will be flat, so the increment will be 0. The decay goes from 1 to zero, so the increment will be gpmag*duraction. Another way to do this would be to use the C_RUNNING generator mode. Envstate will be tested for envelope status: if envstate < 0 then setup an attack, if = 0 then setup a steady state, else setup decay;

```

procedure SET_ENV_AND_DWELL(integer ug; real duration);
begin
  own integer envstate;

```



```
if envstate < 0 then
  begin "ATTACK"
    BIND(ug,exponent,0);           ∂ set up initial point in ramp;
    BIND(ug,asymptote,0);          ∂ set up envelope d.c. offset;
    BIND(ug,rate,gpmag*duration);  ∂ set attack ramp increment;
    envstate=0;
  end "ATTACK"
else if envstate = 0 then
  begin "STEADYSTATE"
    BIND(ug,rate,gpmag*0);  ∂ ramp increment is zero during
                           steady state;
    envstate=1;
  end "STEADYSTATE"
else begin "DECAY"
  BIND(ug,rate,gpmag*-duration);  ∂ ramp increment is negative
                                   during decay;
  envstate=-1;
end "DECAY";
SET_FIELD(dwell,duration*srate);  ∂ set dwell time;
end;
```

Now for the body of the program. Assume the above procedures are put in before the first executable statement.

```
BEGIN "SIMP"
define ∂ = "COMMENT";
∂ get the source file definitions;
Require "LOWER.DEF [SAM,MUS]" source_file;
Require "LOWER.REL [SAM,MUS]" load_module;

integer
simp,                ∂ will contain the address of the sound producing
                    generator;
zero,                ∂ will be given a sum memory addr. not written into;
gen_outa,            ∂ will be the generator in WRITE_DAC mode;
outa,                ∂ will contain a sum memory address connected to a
                    generator in WRITE_DAC mode;
srate,              ∂ will be the sampling rate;
frmag,              ∂ magic number for FREQUENCY;
dfrmag,             ∂ magic number for SWEEP;
gpmag,              ∂ magic number for mumble;
com_chan,brk,eof,fail; ∂ these are for SAIL i/o ;

∂ the above procedures go here;

∂ here begins the program initialization;
simp←GET(id_generator); ∂ gloms a generator and returns its address;

INIT;

SET_GEN(simp,440);                ∂ setup arguments to simp;

SET_ENV_AND_DWELL(simp,2);        ∂ now we get the attack;

∂ here ends the initialization. All parameters must be in place before
any run modes are passed;
BIND(simp,mode,a_running+|plusq+sine);∂ beat me daddy, 8 to the bar...;

SET_ENV_AND_DWELL(simp,2);        ∂ now the steady state;
SET_ENV_AND_DWELL(simp,2);        ∂ now we get the decay;

GIVE(simp);
FINISH;
END "SIMP";
```

5-2. Modifier example

This example presumes the same kind of support code and window dressing as was presented in the last example for setting up DAC channels, clock commands and so on. Here we only see one subroutine called MIX which will configure a modifier to act in MIXING mode. This is a subroutine out of the intermediate level library written at Stanford.

Here is the description of it taken from [Schottstaedt]:

```
MOD←MIX(OUTLOC,IN1LOC,FACTOR1,IN2LOC,FACTOR2);
```

Sets up a modifier to mix two signals by constant factors, each one of which may be between -8 and +8. IN1LOC is the sum memory address where the first signal comes from. It will be scaled by FACTOR1. IN2LOC is the sum memory address where the second signal comes from. It will be scaled by FACTOR2.

This procedure is a single module which can be compiled and loaded separately from any other procedure, allowing the user to select only those routines that are useful to a particular application. The "Entry" statement tells the SAIL compiler that it is to be a separate module. This procedure will call a couple of other procedures, namely GSCALE, GETPE, and BOXERROR, which are other intermediate level procedures. GSCALE is used to convert SAIL type real numbers into a 20 bit integer coefficient and corresponding scale terms for correct modifier arithmetic. GETPE is a fancier version of GET that does some statistics on element usage but is otherwise the same as GET. BOXERROR is actually a high level procedure that has been setup with the low level SETPROCEDURE routine to act as the error handler for all software errors (the actual setup of BOXERROR is done elsewhere).


```

entry MIX;
begin "MIX"
define != "COMMENT";

! MIX - Multiply two signals by a constant factors between 0 and 8.
Returns number of modifier it got.
You have to take care of returning said modifier when you are done!;

external Boolean procedure GSCALE(real factor; reference integer val,scl);
external integer procedure GetPe(integer id);
external procedure boxerror(string errmsg);
require "LOWDEF.HDR [SAM,MUS]" source_file;

Internal integer procedure MIX(integer outloc,in1loc; real factor1;
integer in2loc; real factor2);
begin "MX"
integer mod,scale1,cval1,scale2,cval2;

if gscale(factor1,cval1,scale1) then
boxerror("MIX: First factor out of bounds = "&cvg(factor1));

if gscale(factor2,cval2,scale2) then
boxerror("MIX: Second factor out of bounds = "&cvg(factor2));

mod←GetPe(id_modifier);

bind(mod,mode,m_inactive);
bind_field(mod,coeff0,cval1,left_justified);
bind_field(mod,coeff1,cval2,left_justified);
bind(mod,A_in,in1loc);
bind(mod,B_in,in2loc);
bind(mod,sum_memory,outloc);
bind(mod,B_scale,scale2);
bind(mod,A_scale,scale1);      ! A_scale goes with coeff0;
bind(mod,mode,mixing);

return(mod);
end "MX";
END "MIX";

```

5-3. Delay line example

The following is supplied as a terse example of using a delay line for table lookups. It just shows how to connect delay memory to a delay line to a modifier, and how to fill the delay memory. Left out are all the settings of the modifier parameters and read-write addresses to sum memory and the interface to either the DACs or a file.

```

entry
begin "delay"

define ⌘ = "comment ";

require "LOWDEF.HDR" source_file;
integer delay_port,mod;
define len = "10";
define loc = "1033"; ⌘ some random place in delay memory;
preload_with 0,1,2,3,4,5,6,7,8,9;
safe integer array table[0:len-1];

⌘ LOAD DELAY DOESN'T WORK ANYMORE (never did...);
load_delay(table,loc,len); ⌘ stuff array TABLE into someplace in delay memory;
mod ← get(id_modifier);
delay_port ← get(id_delay);
bind(delay_port,mode,table_lookup); ⌘ set to table lookup mode;
bind(delay_port,base_address,loc); ⌘ this delay starts at loc;
bind(delay_port,delay_length,len); ⌘ len lookups;
bind(delay_port,index,len/2); ⌘ set index to some location or other;
bind(mod,invoke_delay_unit,delay_port); ⌘ delay_port is bound to mod;

⌘ we assume great things happen here;

⌘ now lets change the function of the delay unit to delay line mode;
bind(delay_port,mode,delayline); ⌘ We can now use this as a delay line;

⌘ More great things happen here;

give(delay_port); ⌘ OK, give it back!;
give(mod);

end "delay";

```

5-4. Writing your own procedures

The following is an example of how you could write your own procedures to handle box setup. This routine gets *N* consecutively numbered generators. It looks up array `UNIT_GENERATORS` which contains a location for all the elements in the box and whether they are in use or not. It requires a global variable `NPTIX` which would contain the number of processing ticks available on this pass, and hence the number of processing elements that can run. It also needs to know what kind of processing element you want, specifying it by saying `ID_GENERATOR`, `ID_MODIFIER`, `ID_DELAY` or `ID_SUM_MEMORY` which are defined in `LOWER.DEF` to be 0, 1, 2 and 3 respectively. The routine returns the number of the first unit in the sequence. If there are not *N* units in a row, it returns -1.

```

integer procedure GET_N_UNITS(integer TYPE, NLOCS);
begin "GET_N"
  integer I,J,BASE,LIMIT,NEED_N_TIX;
  define @ = "COMMENT";
  boolean GOTIT;

  @ BASE contains the base address of the type
    of element in UNIT_GENERATORS;
  base← case TYPE of(0,256,384,640);
  @ now figure out how many processing ticks the desired
    number of processing elements would take;
  case type of
  begin
    [0] need_n_tix←nptix; @ generators take only one
      processing tick;
    [1] need_n_tix←nptix/2; @ it takes two processing
      ticks per modifier;
    [2] need_n_tix←nptix; @ delay memory takes no
      processing ticks;
    [3] need_n_tix←(nptix-6)/4 @ this is the formula
      for available delay cycles;
  end;

  @ LIMIT has maximum processing ticks for all of a
    type of element to be running;
  limit← case type of(256,256,0,134);
  if nlocs > need_n_tix then
  begin
    outstr("You are too greedy");
    return(0);
  end;

  gotit←false;
  for i←0 step 1 until NPTIX-NLOCS do
  begin "ISTART"
    if ¬UNIT_GENERATORS[i+base] then
    begin "CHKIT"
      for j←i+1 step 1 until i+NLOCS-1 do
      if UNIT_GENERATORS[j+base] then
      begin "NOGOOD"
        i←j;
        continue "ISTART";
      end "NOGOOD";
      gotit←true;
      done "ISTART";
    end "CHKIT";
  end "ISTART";
  if ¬gotit then return(-1);
  for j←i step 1 until i+NLOCS-1 do
    UNIT_GENERATORS[j+base]←true;
  return(i);
end "GET_N";

```

Appendix 6

Generator field and mode tables

6-1. Generator parameters

ID	SIZE	DEFINITION	FUNCTION
GO	20	SWEEP	oscillator frequency sweep rate
GJ	28	FREQUENCY	oscillator frequency
GK	20	ANGLE	oscillator angle
GN	11	NCOSINES	number of cosines to be summed
GM	4	SCALE	binary scale of sine or sum of cosines
GP	20	RATE	envelope rate of change
GQ	24	EXPONENT	envelope current value
GL	12	ASYMPTOTE	asymptote (DC offset of the envelope)
GSUM	6	SUM_MEMORY	sum memory location into which output is added
GFM	7	FM	sum memory address from which frequency modulation data is taken
GMODE	10	MODE	generator mode, including run mode and type of oscillator and envelope processing (see list of modes on page 20).
		OSC_MODE	sets oscillator field of GMODE without altering run mode or envelope mode.
		ENVELOPE	sets envelope field of GMODE without altering run mode or oscillator mode.

6-2. Generator run modes

Definition	Osc. run?	Envelope run?	Add to sum?
G_INACTIVE	no	no	no
G_PAUSE	no	no	no
G_WAIT	yes	no	no
x_RUNNING:			
A_RUNNING	yes	yes	yes
B_RUNNING	yes	yes	yes
C_RUNNING	yes	yes	yes
DATA_READ	no	yes	yes
DATA_WRITE	no	no	no
DAC_WRITE	no	no	no

6-3. Oscillator modes

SAWTOOTH	sawtooth wave
SQUARE	square wave
SUM_OF_COSINES	sum of cosines
PULSE_TRAIN	pulse train
SINE	sine(ANGLE), normal sine wave mode (with fm).
SIN_FM	sine(FREQUENCY + FM) special sine table lookup mode.

6-4. Envelope modes

Definition Function

LPLUSQ	Add EXPONENT (GQ) to the offset constant in ASYMPTOTE (GL).
LMINUSQ	Subtract EXPONENT (GQ) from offset constant in ASYMPTOTE(GL)
LEXPLUS	Add ASYMPTOTE (GL) to $2^{(-\text{EXPONENT (GQ)})}$ and scale.
LEXMINUS	Subtract ASYMPTOTE (GL) to $2^{(-\text{EXPONENT (GQ)})}$ and scale.

Appendix 7

Modifier field and mode tables

7-1. Modifier parameters

Id	Size	Definition	Function
M0	30	COEFF0	coefficient
M1	30	COEFF1	other coefficient
L0	20	TERM_0	running term
L1	20	TERM_1	other running term
MIN	8	A_IN	read address in sum memory
MRM	8	B_IN	other read address
MSUM	7	SUM_MEMORY	write address in sum memory
		ADD_SUM_MEMORY	same as SUM_MEMORY.
		REPLACE_SUM_MEMORY	also has write addr. but result replaces sum mem. value
MMODE	9	MODE	run mode of modifier and scale factor for the multiplies
		A_SCALE	scales COEFF1.
		B_SCALE	scales COEFF0.

7-2. Modifier procedures

Definition Function

M_INACTIVE inactive.

MIXING fixed binary point result of $\text{COEFF}_0 * A_{\text{IN}} + \text{COEFF}_1 * B_{\text{IN}}$.

INT_MIXING Integer result of $\text{COEFF}_0 * A_{\text{IN}} + \text{COEFF}_1 * B_{\text{IN}}$.

LATCH Result $\leftarrow$ Term₁. If $B_{\text{IN}} * \text{COEFF}_0 \neq 0$ then TERM₁ $\leftarrow$ A_{IN}.

SIGNUM If $A_{\text{IN}} * \text{COEFF}_0 < B_{\text{IN}} * \text{COEFF}_1$ then result $\leftarrow$ -1 else if $A_{\text{IN}} * \text{COEFF}_0 = B_{\text{IN}} * \text{COEFF}_1$ then result $\leftarrow$ 0 else if $A_{\text{IN}} * \text{COEFF}_0 > B_{\text{IN}} * \text{COEFF}_1$ then result $\leftarrow$ 1.

ZERO_CROSSING_PULSER
If $(B_{\text{IN}} * \text{COEFF}_0) * (\text{TERM}_1 * \text{COEFF}_1) \leq 0$ then result $\leftarrow$ - 1
else result $\leftarrow$ 0. TERM₁ $\leftarrow$ B_{IN} * COEFF₀.

MINIMUM The lesser of $A_{\text{IN}} * \text{COEFF}_0$ or $B_{\text{IN}} * \text{COEFF}_1$ is returned.

MAXIMUM The greater of $A_{\text{IN}} * \text{COEFF}_0$ or $B_{\text{IN}} * \text{COEFF}_1$ is returned.

AM
Result $\leftarrow$ TERM₁ * COEFF₁. TERM₁ $\leftarrow$ A_{IN} * ((B_{IN} + 1) / 2).

FOUR_QUAD_MULTIPLY
Result $\leftarrow$ TERM₁ * COEFF₁. TERM₁ $\leftarrow$ A_{IN} * B_{IN}.

U_NOISE TERM₁ $\leftarrow$ Result $\leftarrow$ COEFF₀ * TERM₁ + TERM₀.

TR_U_NOISE Result $\leftarrow$ COEFF₀ * TERM₁ + TERM₀.
If $B_{\text{IN}} * \text{COEFF}_1 \neq 0$ then TERM₁ $\leftarrow$ result.

THRESHOLD If $\text{COEFF}_0 * A_{\text{IN}} + \text{TERM}_0 < 0$ then result $\leftarrow$ 0, else
if ≥ 0 then result $\leftarrow$ COEFF₁ * B_{IN}.

FILTERING:
ONE_POLE
ONE_ZERO
TWO_0POLES COEFF₀ variable
TWO_1POLES COEFF₁ variable
TWO_0ZEROS COEFF₀ variable
TWO_1ZEROS coeff₁ variable

Appendix 8

Delay unit field and mode tables

Fields:

Id	Definition	Function

P	MODE	Delay unit mode
Z	DELAY_LENGTH	Delay line unit length
	SCALE	or table address scale factor
Y	INDEX	Running index of the delay array
X	BASE_ADDRESS	Base address of delay array

Modes:

D_INACTIVE	dead
DELAYLINE	
TABLE_LOOKUP	
ROUNDED_TABLE_LOOKUP	

8-1. Delay units: timing

Delay units have their own formula: Delay memory cycles available per pass =

$$(\langle \#_of_processing_ticks \rangle - 6) / 4. \quad \text{Equ. 23}$$

(If you are loading delay memory while doing processing, i.e., if you are changing delay memory arrays from the main computer during execution of samples, then the number of delay cycles is this number less however many cycles the main computer may take to load the array. Each delay memory cycle used to load arrays in this fashion will load one word. This method probably will not be used much. The typical usage will be to load the arrays before cranking up the box, so that the sound processing is not in competition with computer I/O.)

Appendix 9

Reserved Words

a_in	data_write
a_running	decode
a_scale	delay
add_sum_memory	delay_length
address	delayline
all_sum_memory	delays
am	diagnostic_address
angle	dis_ticks
asymptote	dwel
b_in	dx
b_running	dx_load
b_scale	ena_ticks
bad_linger	envelope
base_address	envlsb
bias_delay	envmask
bias_end	exponent
bias_generator	f0
bias_modifier	f1
bias_sum_memory	f2
bind	f3
bind_field	flush
c_exhausted	flusher
c_running	fm
c_start	fn_maximum
c_step	fn_minimum
c_stop	fnlsb
clear_passes_dwell	fnmask
code_stream	four_quad_multiply
coeff0	frequency
coeff1	full_word
command_stream	function
control_mode	g_inactive
d_base	g_pause
d_inactive	g_wait
d_index	gen_last_pass_sum_memory
d_mode	gen_this_pass_sum_memory
d_scale	generators
d_size	get
dac_write	give
data_read	gsum_overflow

id_delay
id_generator
id_maximum
id_modifier
id_sum_memory
index
inhibit_processing_ticks
initialize
initialize
int_mixing
invoke_delay_unit
last_gen_pass
last_mod_pass
latch
left_justified
lexpminus
lexpplus
linger
lminusq
lounge
lplusq
m_inactive
max_envelope
max_oscillator
max_sfield
max_stream
maximum
maxPass
min_envelope
min_oscillator
minimum
mix_overflow
mixing
mod_last_pass_sum_memory
mod_this_pass_sum_memory
mode
modifiers
msum_overflow
mult_overflow
ncosines

non_optimize
notwopoles
notwozeroes
one_pole
one_zero
optimize
osc_mode
osclsb
oscmask
packing_mode
pass
pass_clear
pass_set
pause_clear
permit_processing_ticks
PIA
processing_ticks
pulse_train
putcmd
r_32_data
r_exhausted
rate
relative
replace_sum_memory
reset
reset_tick_counter
right_justified
round_table_lookup
run_mode
runlsb
runmask
same_filter
sawtooth
scale
set_code
set_field
set_interrupt
set_mode
set_output
set_passes

<[Set_mode: Mode index out of range> <[Set_field: Field index out of range> <[Get: Identifier
 type out of range.> <[Get: Memory quadrant illegal without memory id.> <[Get: Warning- can't
 get requested element.> <[Give: Identifier out of range - > <[Bind: Identifier out of range.>
 <[Bind: Illegal parameter type for (parameter so-and-so)> <[Flush: No output channels!> <[Bind:
 too many bits for (parameter in question)>
 set_stream
 sig (processing element in question)
 sin_fm
 Value = (value, printed in octal and decimal integers)> <[Bind: Attempt to write (modifier or
 size buffer sum)>
 square
 from (processing element type and number)> <[Bind: Attempt to use (processing element type and
 number) memory
 sum_of_cosines
 sweep (input, either FM, A_IN or B_IN)> <[Bind: Attempt to use (processing element type and
 number) pickup
 term_0
 as delay unit for modifier (number)> <[Set_output: Not code or command stream> <[Set_output:
 illegal channel> <[Set_output: Too many channels> <[Unset_stream: bad stream number>
 if Set mode pass Bad stream number> <[Set_stream: Bad field>
 threshold
 tix_processing
 tix_total
 total_ticks
 tr_u_noise
 ttl_load
 ttla
 ttlb
 two_0poles
 two_0zeroes
 two_1poles
 two_1zeroes
 u_noise
 unfiltered
 unit_generators
 unset_output
 update_tick
 w_exhausted
 wait_clear
 zero_crossing_pulser

DATE!

DRAFT

Appendix 10

Error messages

Appendix 11

References

[Gosper], William, Numerical Experiments with the Spectral Test, STAN-CS-75-490.

[Knuth], Donald, The Art of Computer Programming, Vol. II. (Chapter 3).

[Harvey], Brian, Monitor Command Manual, Stanford Artificial Intelligence Laboratory, Operating Note 54.6. Fourth ed., August, 1978.

[Moorer], James A., Principles of the Intermediate Level Software for the Systems Concepts Digital Synthesizer, ~~unpublished manuscript, Center for Computer Research in Music and Acoustics, Stanford University, Stanford, CA.~~

[Samson], Peter, Systems Concepts Digital Synthesizer Programming Specification, a publication of Systems Concepts, 3rd. St., San Francisco, CA.

[Schottstaedt], et. al., Description of Intermediate Level Library Routines for the Systems Concepts Digital Synthesizer, ~~in-house publication, Center for Computer Research in Music and Acoustics, Stanford University, Stanford, CA. 94301.~~

[Winham] and Steiglitz, "Input Generators for Digital Sound Synthesis", J. Acoust. Soc. Am., Volume 47, #2 (part 2) 1970, pp665-666.

I N D E X

(References are to Page numbers)

A_IN	41, 46	Command stream	12
A_RUNNING	21	command underrun	14
A_SCALE	42	CONO	69
ADD_SUM_MEMORY	43		
Addressing sum memory	16	DAB	84
AM	47	DAC	33
Amplitude modulation	47	DAC stream	13
Analog Filters	66	DAC_WRITE	22
ANGLE	25, 34	DECODE	76
Arithmetic overflow	21, 22, 49	Delay command	67
array all_sum_memory	75	Delay line example	109
array delays	75	Delay line mode	61
array gen_sum_memory	75	delay memory	2, 9, 61, 62, 63
array gen_this_pass_sum_memory	75	Delay unit	2
array generators	75	Delay unit field and mode tables	116
array mod_sum_memory	75	Delay units	9, 61, 62, 63
array modifiers	75	Delay units: timing	116
array unit_generators	75	DELAYLINE	62
Assembler tick and pass counters	80	digital filtering theory	54
ASYMPTOTE	25, 30, 35	direct memory access	13
		Disassembly	84, 85
B_IN	41, 46	DMA	13, 22
B_RUNNING	21	DX	66
B_SCALE	42		
band limiting	29	envelope	8, 18
band-limited pulse train	29	Envelope modes	32
bandwidth	55	Envelope processing	30, 31
base address	76	Error messages	121
Basic number theory	97, 98, 99, 100	exiting	82
Bind	77	EXPONENT	30, 31, 35
BIND_FIELD	78		
block structure	93	flow of data	12, 13, 14
		FLUSH	82
C_RUNNING	22	FM	22, 25, 35
clear all pause bits	20, 66	FM scaling	38
clear all wait bits	21, 66	foldover	29
CLEAR_PASSES_LINGER	67	formant filter	55
COEFF0	42	Four quadrant multiply	47
COEFF1	42	FOUR_QUAD_MULTIPLY	47
Coefficient terms	42	Free mode	21
Command assembly	77, 78, 79, 80, 81	FREQUENCY	25, 33
Command mnemonics	9	frequency response	55
command overrun	14, 67	Frequency scaling	36

Function	43	LINGER	14, 67
G_PAUSE	20	LMINUSQ	32
G_WAIT	21, 22	Low level allocation routines	73, 74, 75, 76
General definitions	2	Low level routines	71
generator	2, 95	low-pass filter	55
Generator field and mode tables	112, 113	LPLUSQ	32
Generator magic numbers	36, 37, 38, 39, 40	M_INACTIVE	46
Generator parameters	18	Maximum	48
Generator running modes	20, 21, 22, 23	MERGE	85
Generator tick time	6	Minimum	48
Generators	8	Miscellaneous command	66
GET	73	Mixing	46
GFM	35	Mode	43
GIVE	74	modifier	2, 95
GJ	33	Modifier field and mode tables	114, 115
GK	34	Modifier filtering algorithms	52, 53, 54, 55, 56, 57, 58, 59
GL	30, 35	Modifier parameters	41
GM	34	Modifier procedures	115
GN	34	Modifier scaling	42
GO	33	Modifier tick time	6
GP	30, 35	Modifiers	8, 41, 42, 43, 44, 45
GQ	30, 35	NCOSINES	34
GSUM	19, 35	Numbering of elements and ticks	102
high-pass filter	55	One pole	52, 54
I/O bus stream	12	One zero	52, 55
INACTIVE	20, 46	oscillator	8, 18
INITIALIZE	82	Oscillator modes	27, 28, 29
input port	16	output port	16
Instruction and command structure	64	Overhead ticks	3
INT_MIXING	46	pass	2, 81, 94, 96, 101
Integer mixing	46	pass counter	67
Interrupt	14	PASS_CLEAR	67
Introduction to Generators	18, 19	pipelining	93, 94
Invoke delay unit	51	Processing element	2
INVOKE_DELAY_UNIT	51	Processing element commands	65, 66, 67, 68
last pass	15	Processing element tables	75
Latch	47	processing elements	8, 9, 10, 95
LEXMINUS	32	processing ticks	102
LEXPLUS	32		

PROCESSING_TICKS	68	Stop clock	66
pulse train	29	Stream output	82, 83
PULSE_TRAIN	28, 29	Stream processing	14
		sum memory	2, 9
RATE	25, 30, 35	SUM_MEMORY	22, 35, 43
Read stream	13	SUM_OF_COSINES	29, 34
Read terms	41	SWEEP	22, 25, 33
READ_DATA	22	Synthesizer instructions	69, 70
read/write data underrun	14		
Real time	101, 102	Table lookup mode	61, 109
Reassembly	84, 85	TABLE_LOOKUP	62
References	122	TERM_0	41, 42, 49
refreshing	25	TERM_1	41, 42, 47
RELATIVE	76	this pass	15
REPLACE_SUM_MEMORY	43	Threshold	48
Reserved Words	118, 119, 120	tick	2, 96
Resonant filter scaling	55	tick-time	3
resonant frequency	55	Ticks	3, 4, 5
resonator	55	Ticks command	68
Ring modulation	47	Time division multiplexing	95, 96
ROUND_TABLE_LOOKUP	62	Timer command	67
Running terms	42	total ticks	3
		TOTAL_TICKS	68
SAIL examples	103, 104, 105, 106, 107, 108, 109, 110, 111	TR_U_NOISE	50
sample	1, 2	Trigger mode	22
sample period	2	Triggered uniform noise	50
sampling rate	1, 2	TTL_A	66
SAWTOOTH	27, 29	TTL_B	66
SCALE	34	Two poles	53, 55
Scale terms	42	Two poles COEFF0 variable	53
SET_FIELD	79	Two poles COEFF1 variable	53
SET_MODE	79	two quadrant multiply	47
SET_OUTPUT	82	Two zeros	53
SET_PROCEDURE	83	Two zeros COEFF0 variable	54
Signum	48	Two zeros COEFF1 variable	54
SIN_FM	28, 34	U_NOISE	49
SINE	27, 29	UNDAB	85
slewing	4	Uniform noise	49
Specifying frequencies to generators	36	update ticks	102
Speed of processing	101	UPDATE_TICK	81
SQUARE	28, 29		
Sticky mode	30	Write stream	13

WRITE_DAC	33, 34
WRITE_DATA	23, 34
Writing your own procedures	110
x_RUNNING	21
ZERO_CROSSING_PULSER	49
Zero-crossing pulser	49